

Vectorized Linear Regression

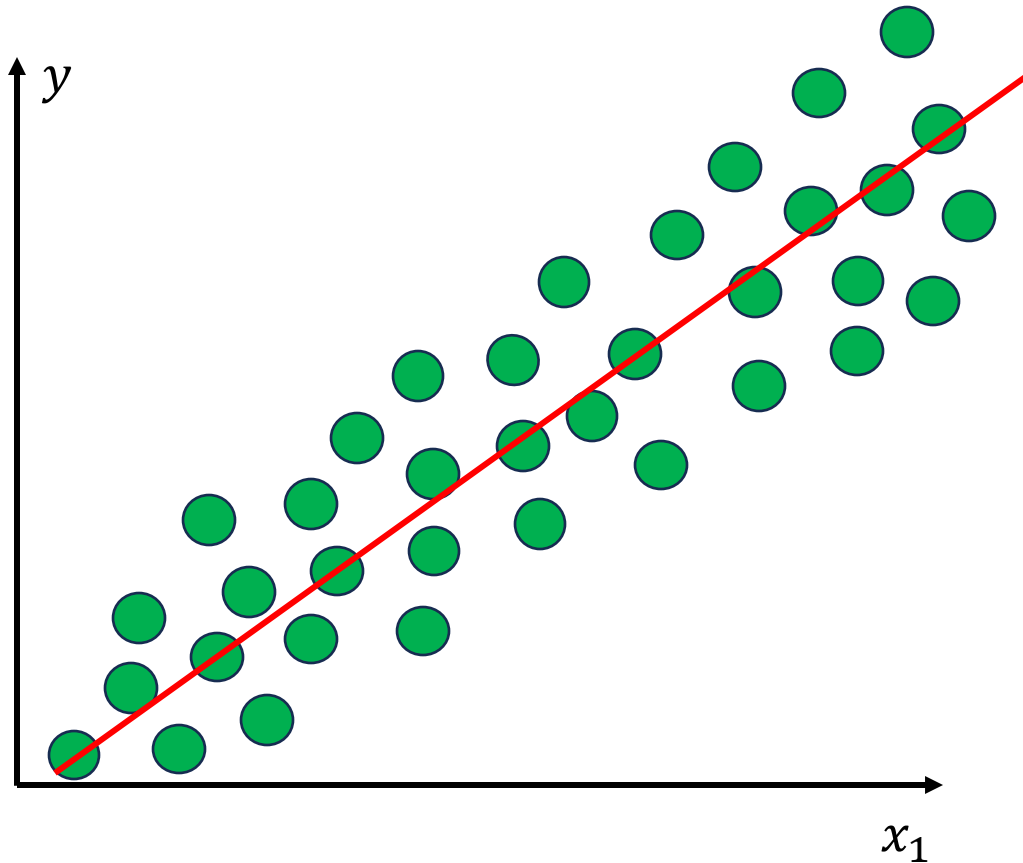
Extra



Dinh-Thang Duong – TA
Yen-Linh Vu – STA

Getting Started

❖ Objectives



Our objectives:

- Revise the simple Linear Regression.
- Discuss a limitation of Simple Linear Regression implementation.
- Introduction to Vectorized Linear Regression.
- Implement the vectorized version of Linear Regression using Gradient Descent.

Outline

- Introduction
- Vectorized Linear Regression
- Code Implementation
- Question

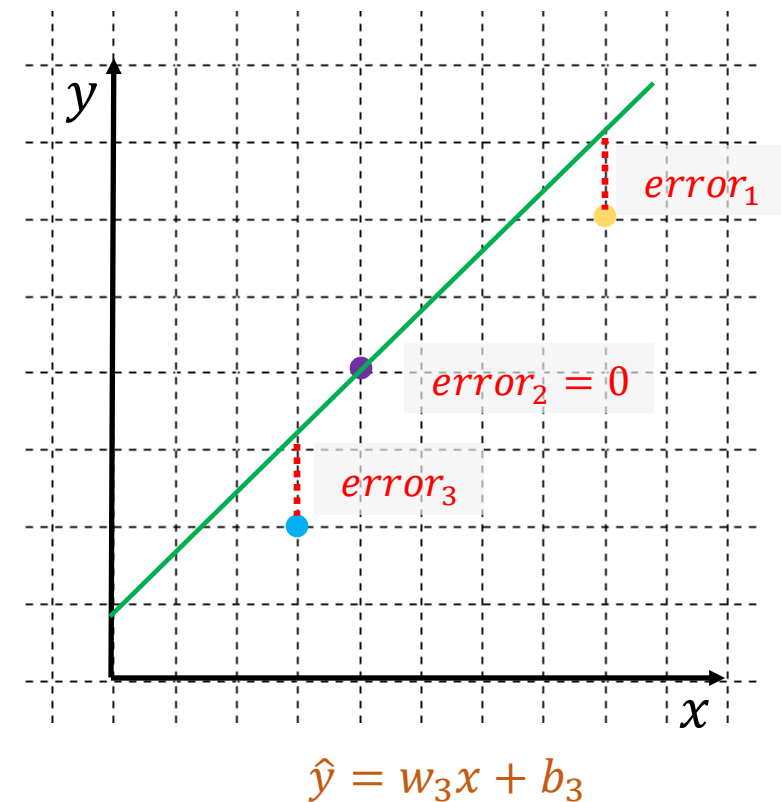
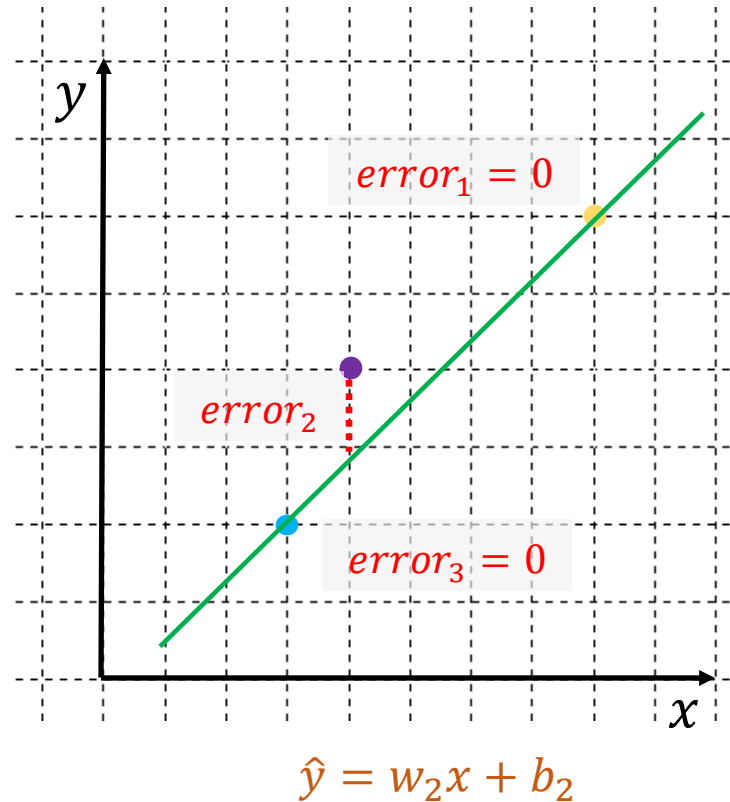
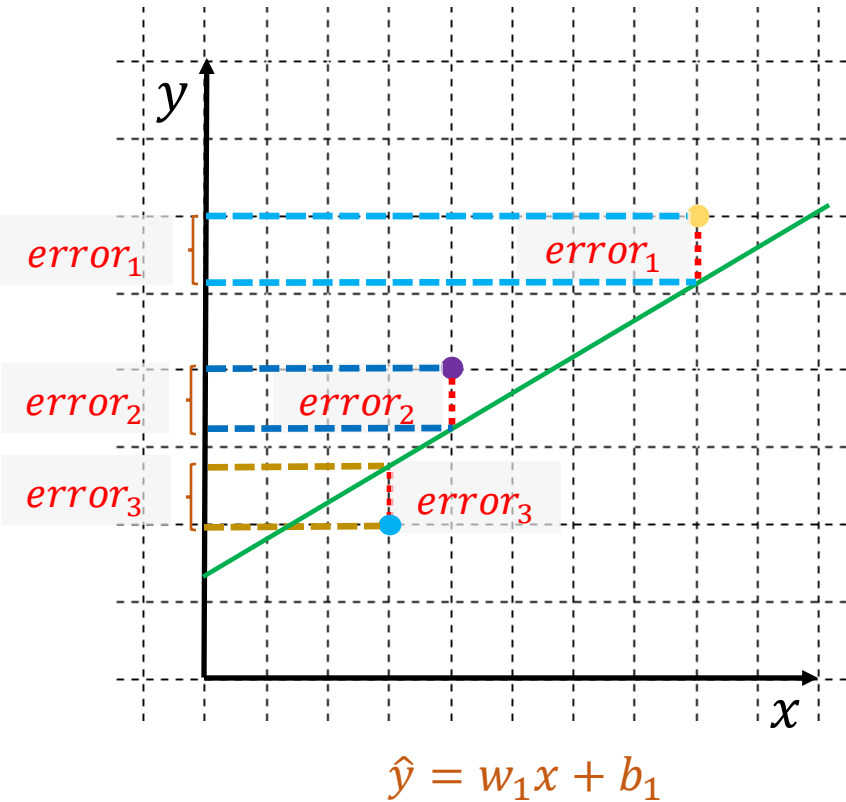
Introduction

Introduction

❖ Recap: Simple Linear Regression

● Training data

$$error_i = distance(\hat{y}_i, y_i)$$



Find w and b whose model has the smallest error, where $loss = \sum_i error_i$

How?

Introduction

❖ Recap: Simple Linear Regression

Linear equation

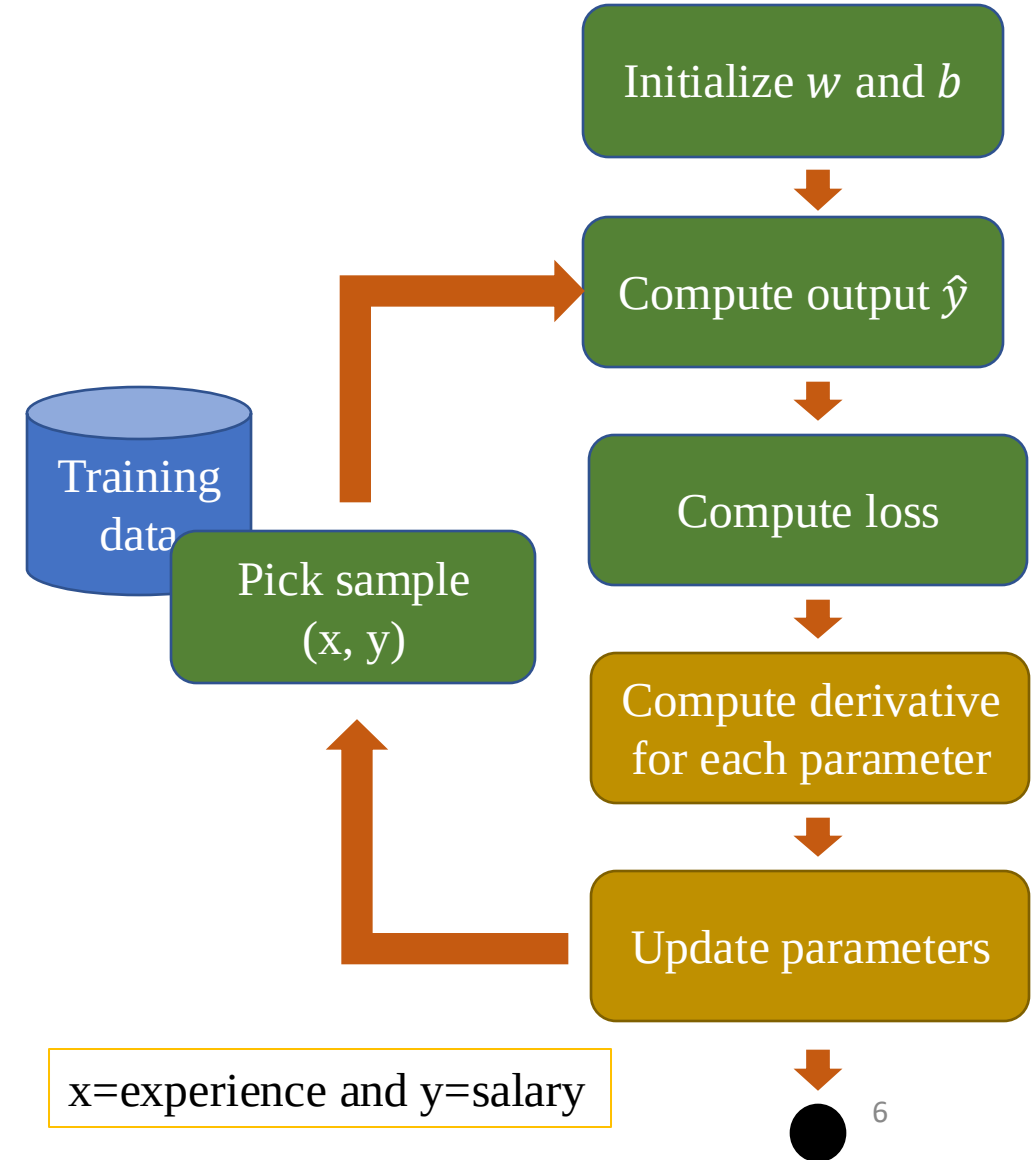
$$\hat{y} = wx + b$$

where $\hat{y}$ is a predicted value,
 w and b are parameters
and x is input feature

Error (loss) computation

Idea: compare predicted values $\hat{y}$ and label values y
Squared loss

$$L(\hat{y}, y) = (\hat{y} - y)^2$$



Introduction

❖ Recap: Simple Linear Regression

Linear equation

$$\hat{y} = wx + b$$

where $\hat{y}$ is a predicted value,

w and b are parameters

and x is input feature

Error (loss) computation

Idea: compare predicted values $\hat{y}$ and label values y

Squared loss

$$L(\hat{y}, y) = (\hat{y} - y)^2$$

Find better w and b

Use gradient descent to minimize the loss function

Compute derivate for each parameter

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w} = 2x(\hat{y} - y)$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial b} = 2(\hat{y} - y)$$

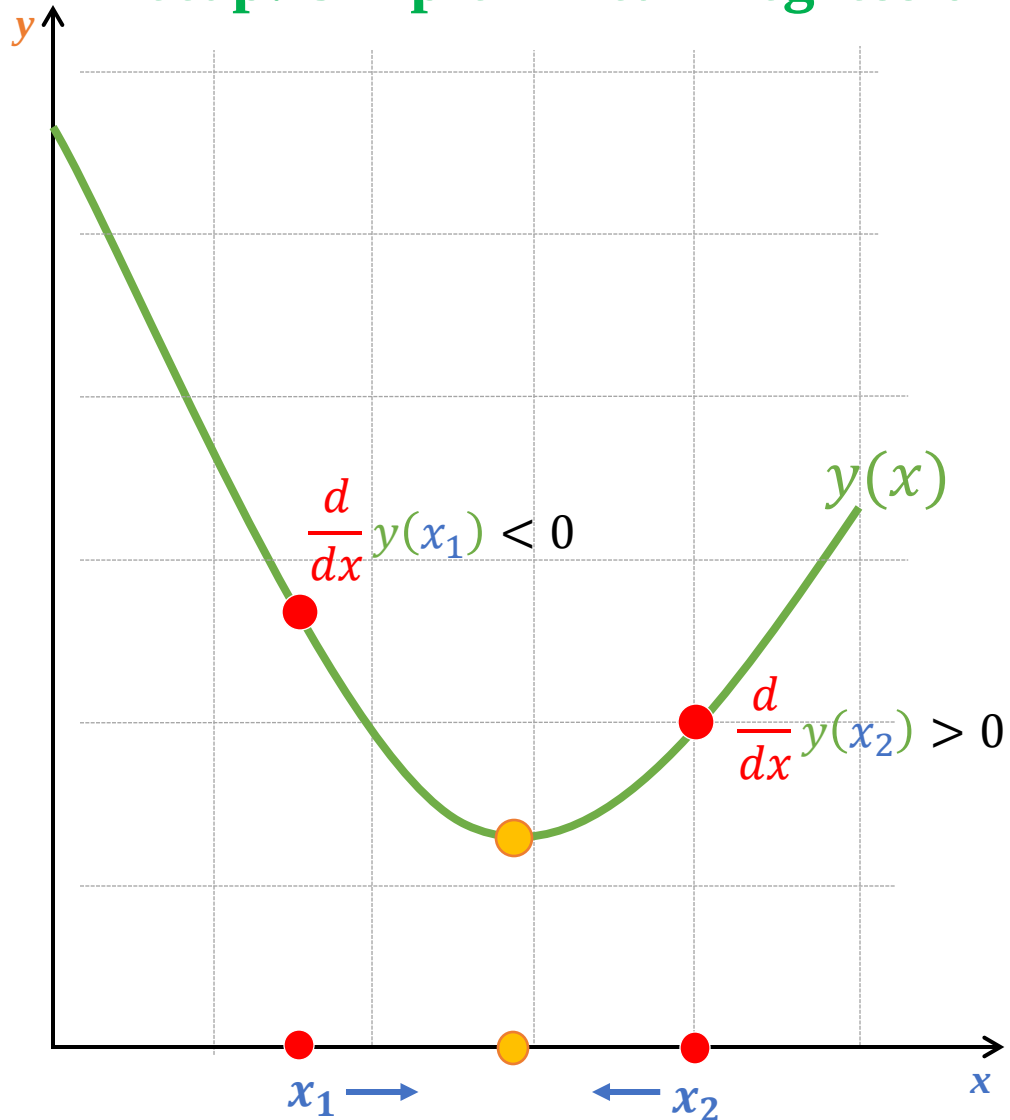
Update parameters

$$w = w - \eta \frac{\partial L}{\partial w} \quad b = b - \eta \frac{\partial L}{\partial b}$$

η is learning rate

Introduction

❖ Recap: Simple Linear Regression



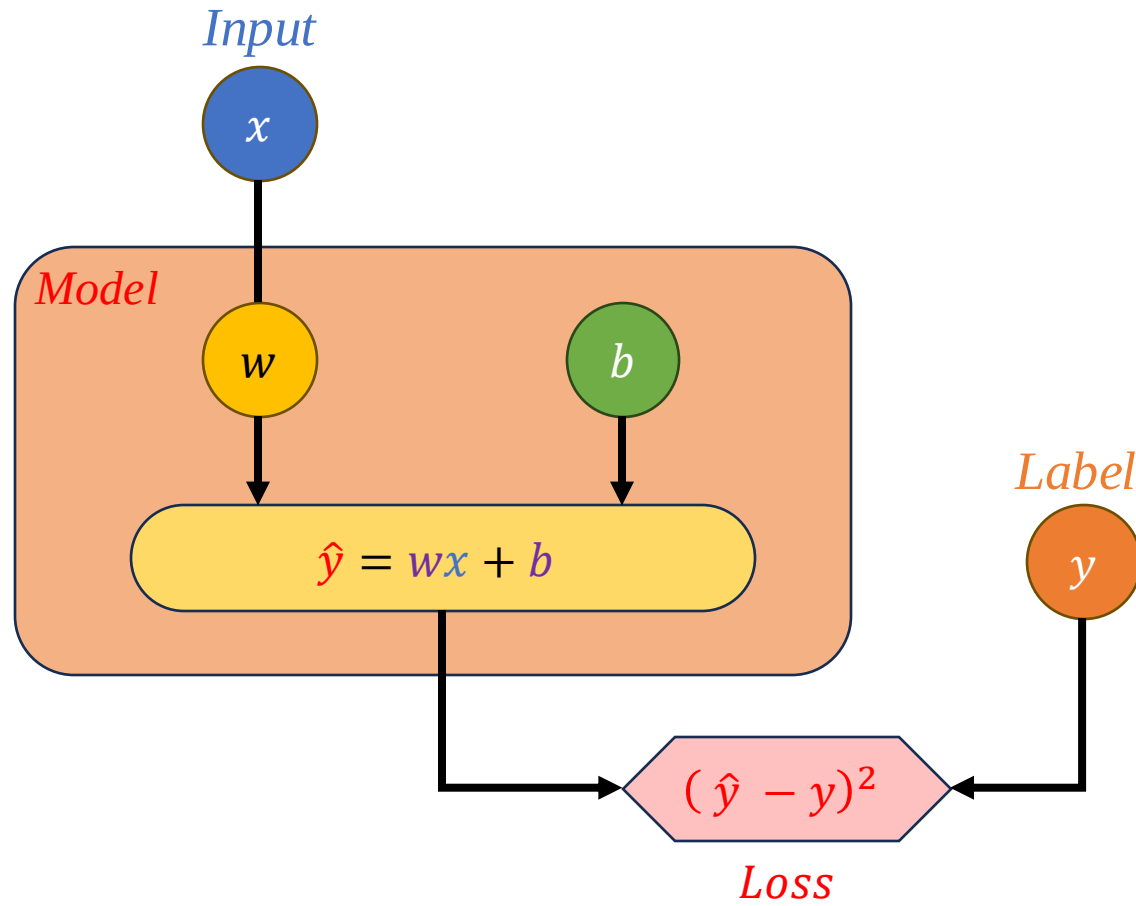
$$\frac{d}{dx}y(x) = \lim_{\Delta x \rightarrow 0} \frac{y(x + \Delta x) - y(x)}{\Delta x}$$

$$x_{new} = x_{old} - \eta \frac{d}{dx}y(x_{old})$$

learning rate

Introduction

❖ Recap: Simple Linear Regression



$$\hat{y} = (wx + b)$$

$$\text{Loss} = (\hat{y} - y)^2$$

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$$

$$b_{\text{new}} = b_{\text{old}} - \eta \frac{\partial L}{\partial b}$$

$$\frac{\partial L}{\partial w} = 2x(\hat{y} - y)$$

$$\frac{\partial L}{\partial b} = 2(\hat{y} - y)$$

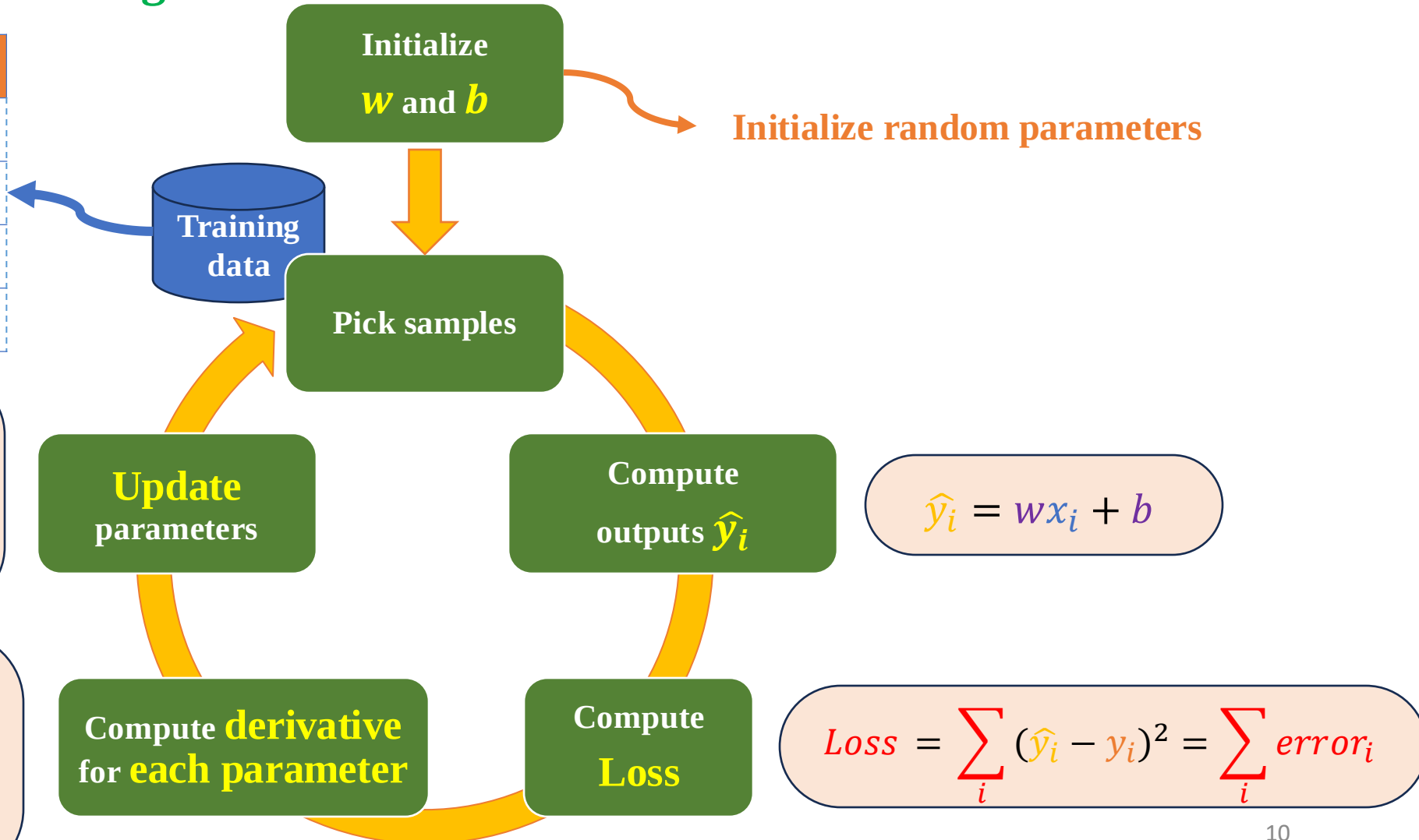
Introduction

❖ Recap: Simple Linear Regression

	Experience	Salary
1	3	60
2	4	55
3	5	66
4	6	93

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$$
$$b_{new} = b_{old} - \eta \frac{\partial L}{\partial b}$$

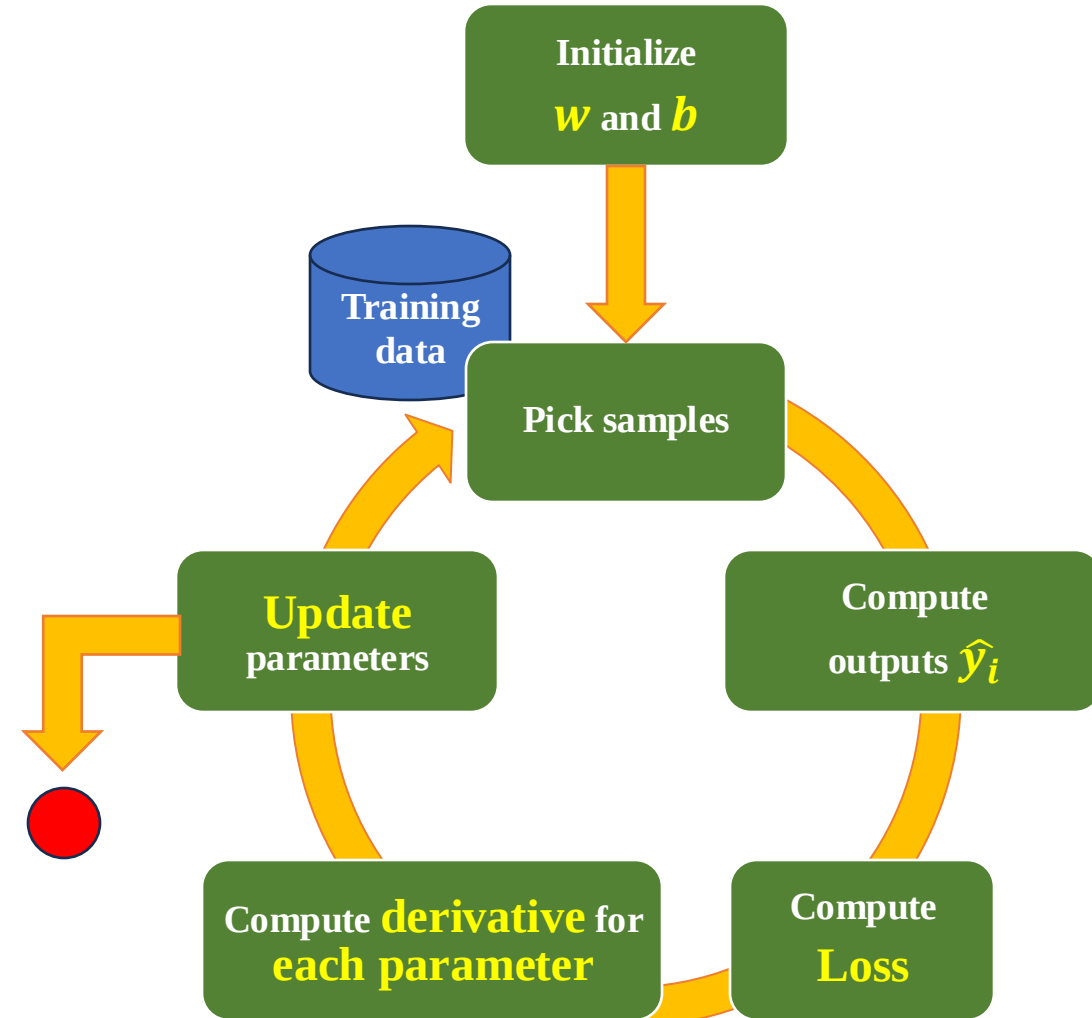
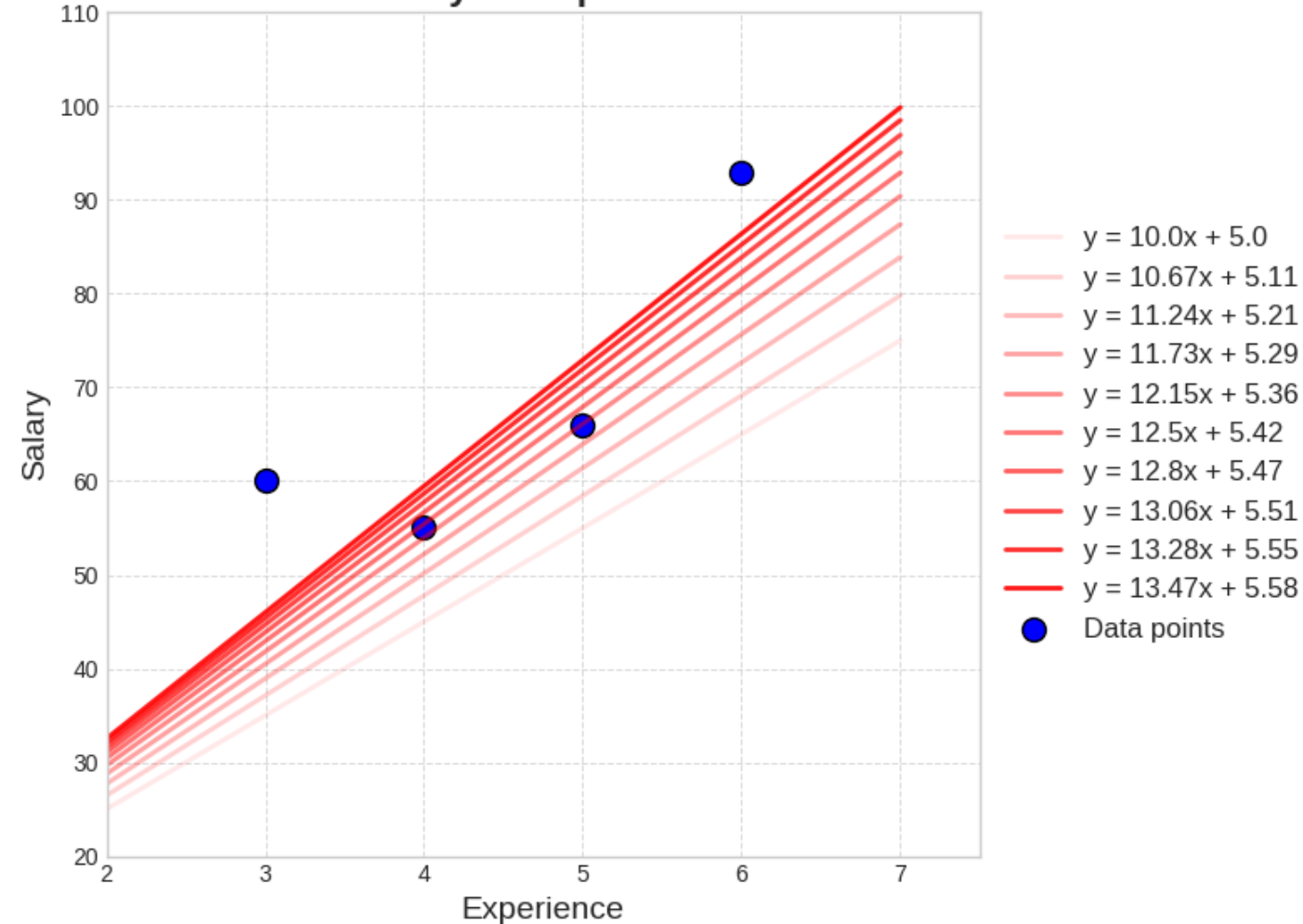
$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial b} = 2(\hat{y} - y)$$
$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w} = 2x(\hat{y} - y)$$



Introduction

❖ Recap: Simple Linear Regression

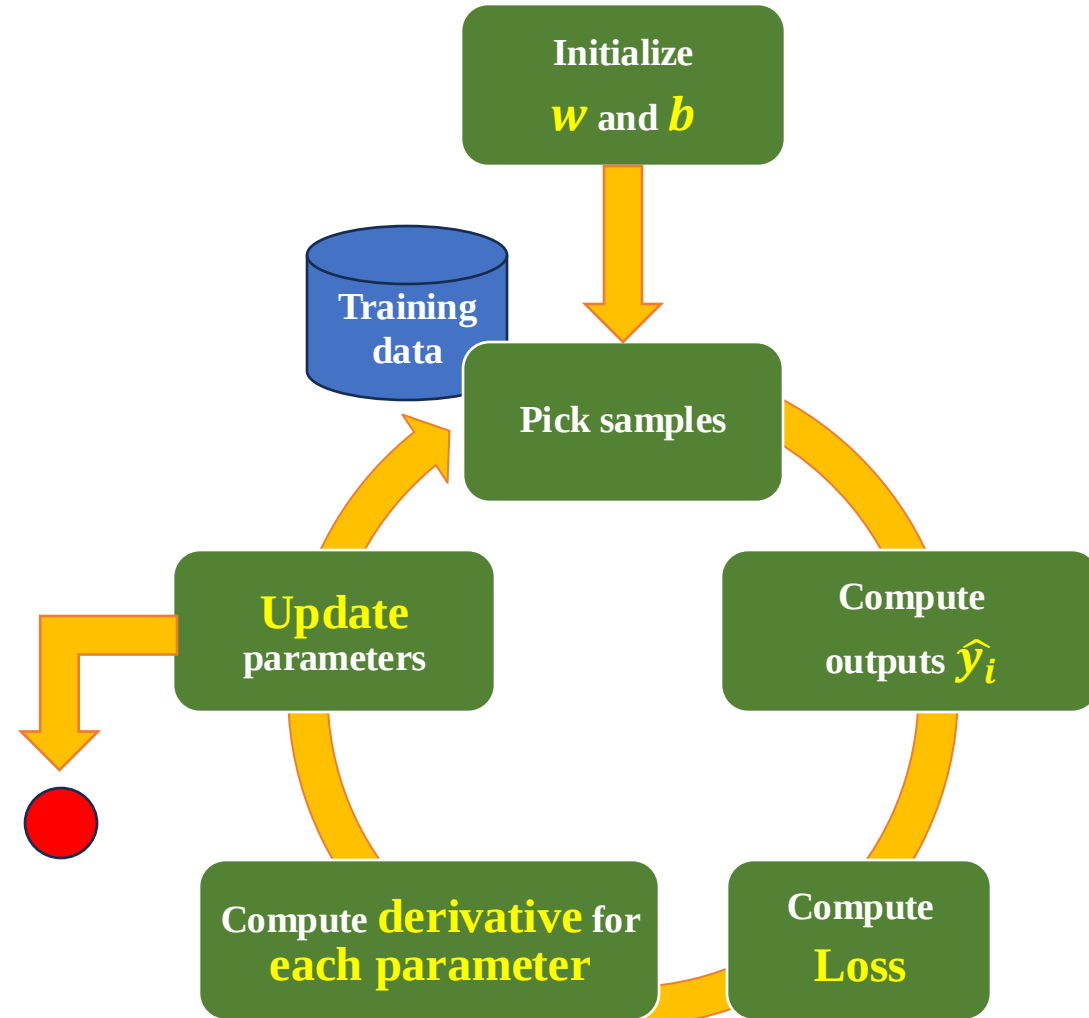
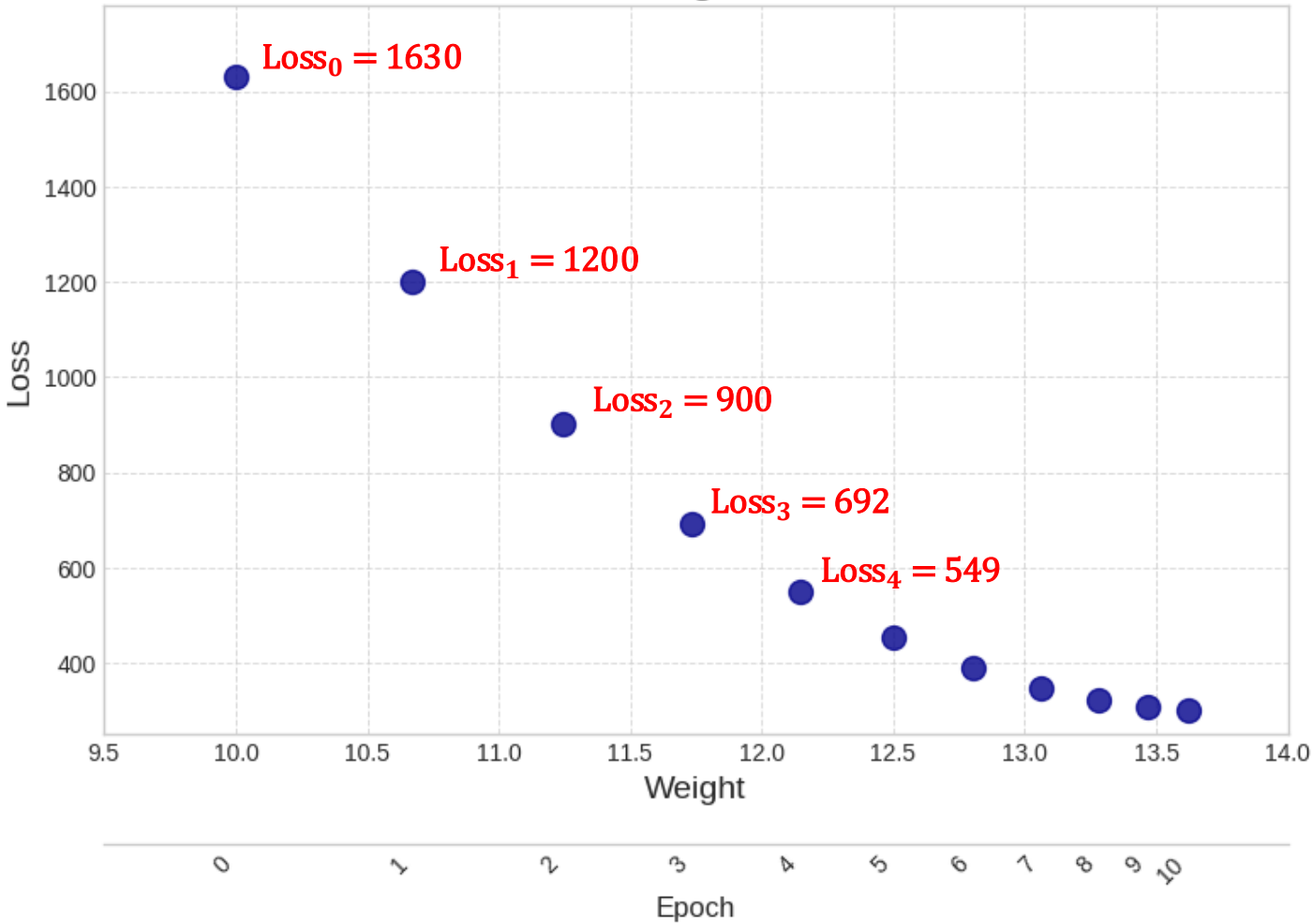
Salary vs Experience



Introduction

❖ Recap: Simple Linear Regression

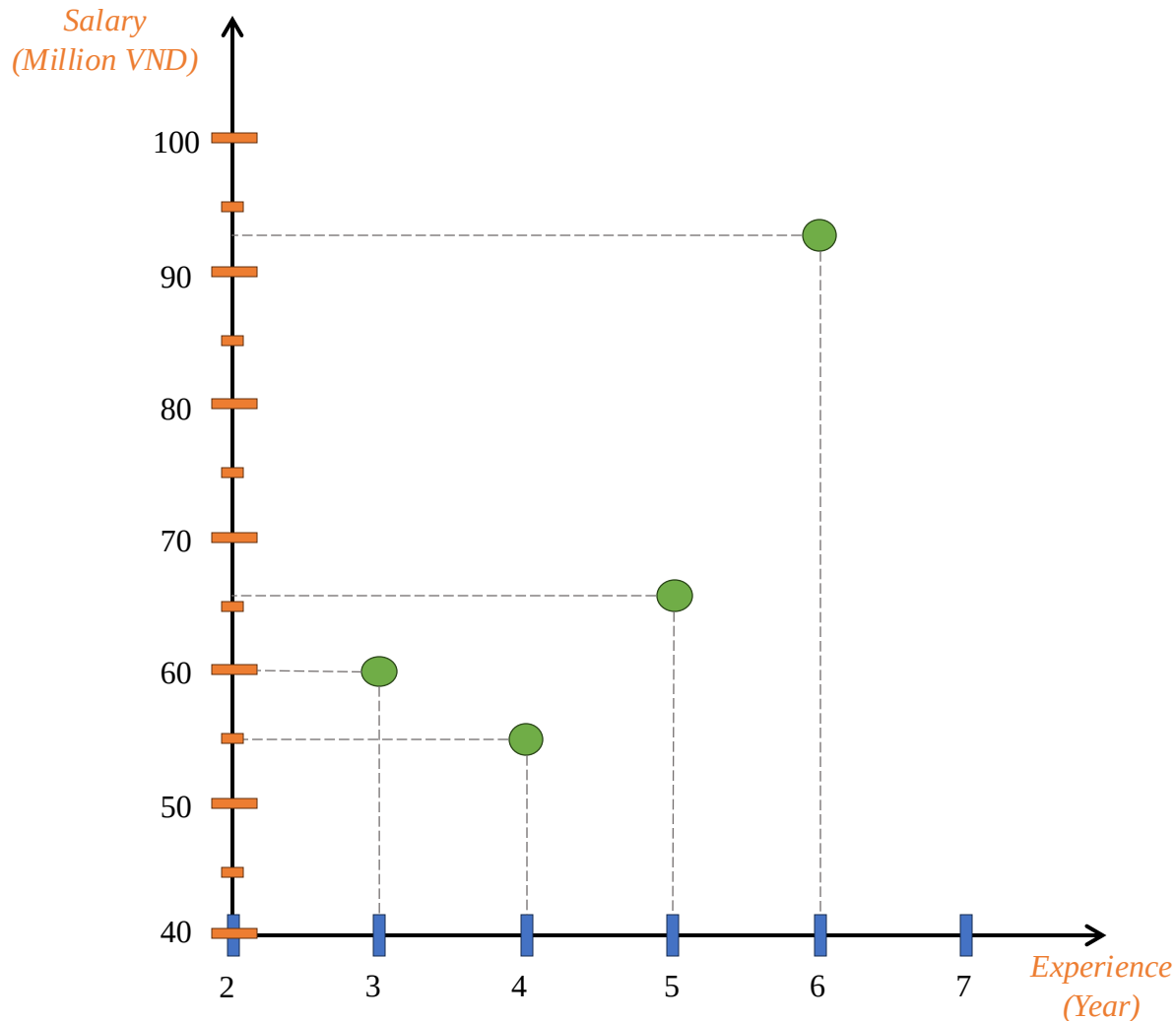
Loss vs Weight and Bias



Vectorized Linear Regression

Vectorized Linear Regression

❖ Univariate Dataset



x

Experience	Salary
3	60
4	55
5	66
6	93
7	?

Vectorized Linear Regression

❖ Univariate Dataset

x

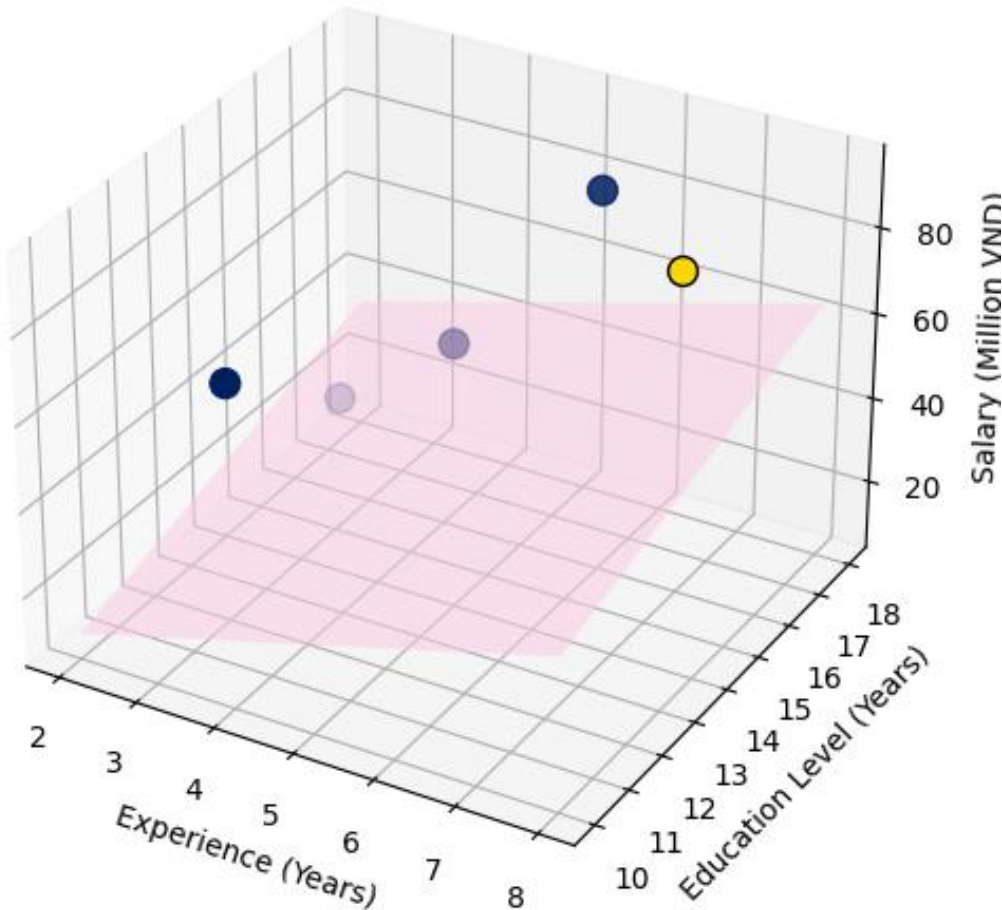
Experience	Salary
3	60
4	55
5	66
6	93
7	?

```
1 def predict(x, w, b):
2     return x * w + b
3
4 def compute_loss(y_hat, y):
5     return (y_hat - y) ** 2
6
7 def compute_gradient(y_hat, y, x):
8     dw = 2 * x * (y_hat - y)
9     db = 2 * (y_hat - y)
10
11     return dw, db
12
13 def update_weight(w, b, dw, db, lr):
14     new_w = w - lr * dw
15     new_b = b - lr * db
16
17     return new_w, new_b
```

Vectorized Linear Regression

❖ Multivariate Dataset

Multivariate Linear Regression: Experience, Education, Salary

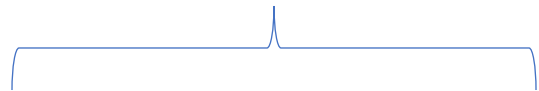


Experience	Education	Salary
3	12	60
4	13	55
5	14	66
6	15	93
7	16	?

Vectorized Linear Regression

❖ Multivariate Dataset in Simple Linear Regression

x_1, x_2



Experience	Education	Salary
3	12	60
4	13	55
5	14	66
6	15	93
7	16	?

The implementation must manually update when the number of input features is changed.

```
1 def predict(x1, x2, w1, w2, wb):
2     return x1 * w1 + x2 * w2 + b
3
4 def compute_loss(y_hat, y):
5     return (y_hat - y) ** 2
6
7 def compute_gradient(y_hat, y, x1, x2):
8     dw1 = 2 * x1 * (y_hat - y)
9     dw2 = 2 * x2 * (y_hat - y)
10    db = 2 * (y_hat - y)
11
12    return dw1, dw2, db
13
14 def update_weight(w1, w2, b, dw1, dw2, db, lr):
15     new_w1 = w1 - lr * dw1
16     new_w2 = w2 - lr * dw2
17     new_b = b - lr * db
18
19     return new_w1, new_w2, new_b
```

Vectorized Linear Regression

❖ Recap: Vector

$x \sim (1)$

5

Scalar

$x \sim (4)$

5
9
2
1

Vector

$x \sim (4, 4)$

5	6	7	9
9	8	7	6
2	3	4	5
1	2	3	4

Matrix

Vectorized Linear Regression

❖ Recap: Vector

$x \sim (4, 5)$

5	6	7	8	9
9	8	7	6	5
2	3	4	5	6
1	2	3	4	5

Matrix

$x \sim (5, 4)$

5	9	2	1
6	8	3	2
7	7	4	3
8	6	5	4
9	5	6	5

Transpose of a matrix

$x \sim (4, 4)$

5	6	7	8
9	8	7	6
2	3	4	5
1	2	3	4

Square Matrix

Exercise 01

❖ Recap: Vector

Some matrix properties:

- $AB \neq BA$ $(A \in \mathbb{R}^{m \times m}, B \in \mathbb{R}^{m \times m})$
- $(rA)^T = rA^T$ $(A \in \mathbb{R}^{m \times N})$
- $(A + B)^T = A^T + B^T$ $(A \in \mathbb{R}^{m \times n}, B \in \mathbb{R}^{m \times n})$
- $(AB)^T = B^T A^T$ $(A \in \mathbb{R}^{m \times n}, B \in \mathbb{R}^{n \times k})$

$$A + B = B + A$$

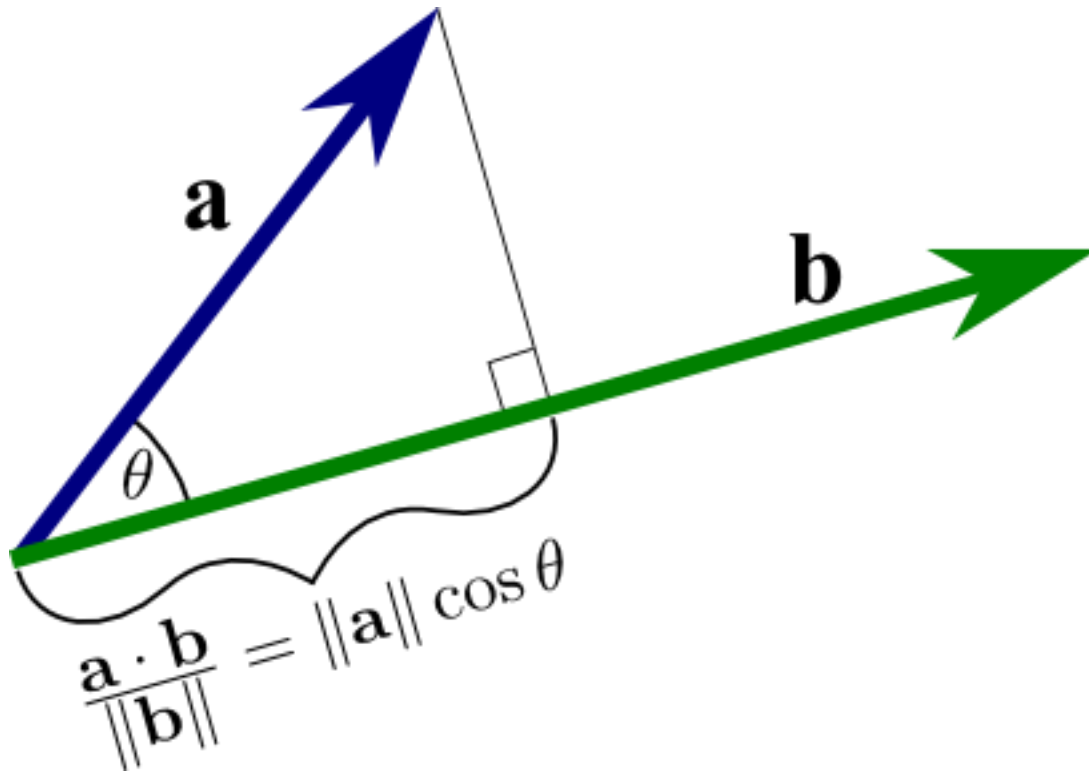
$$A = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{bmatrix}, \quad B = \begin{bmatrix} b_{11} & \cdots & b_{1n} \\ \vdots & \ddots & \vdots \\ b_{m1} & \cdots & b_{mn} \end{bmatrix}$$

$$A \in \mathbb{R}^{m \times n}, B \in \mathbb{R}^{m \times n}$$

$$A + B = \begin{bmatrix} (a_{11} + b_{11}) & \cdots & (a_{1n} + b_{1n}) \\ \vdots & \ddots & \vdots \\ (a_{m1} + b_{m1}) & \cdots & (a_{mn} + b_{mn}) \end{bmatrix} = \begin{bmatrix} (b_{11} + a_{11}) & \cdots & (b_{1n} + a_{1n}) \\ \vdots & \ddots & \vdots \\ (b_{m1} + a_{m1}) & \cdots & (b_{mn} + a_{mn}) \end{bmatrix} = B + A$$

Vectorized Linear Regression

❖ Recap: Dot Product



- $\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta$
- $\mathbf{a} \cdot \mathbf{b} = \sum_{i=1} a_i b_i$

- $\mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$

- $\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2 + a_3 b_3$

Vectorized Linear Regression

❖ Recap: Dot Product Example

$$\begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{bmatrix} \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \\ b_5 \\ b_6 \\ b_7 \\ b_8 \\ b_9 \end{bmatrix} = \begin{bmatrix} c_1 & c_2 & c_3 \\ c_4 & c_5 & c_6 \\ c_7 & c_8 & c_9 \end{bmatrix}$$

Vectorized Linear Regression

❖ Vectorized Linear Regression: Conception

1) Pick a sample (x, y) from training data

2) Compute the output $\hat{y}$

$$\hat{y} = wx + b$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\frac{\partial L}{\partial w} = 2x(\hat{y} - y)$$

$$\frac{\partial L}{\partial b} = 2(\hat{y} - y)$$

5) Update parameters

$$w = w - \eta \frac{\partial L}{\partial w}$$

$$b = b - \eta \frac{\partial L}{\partial b}$$

η is learning rate

$$\hat{y} = wx + b \quad x = \begin{bmatrix} 1 \\ x \end{bmatrix} \quad \theta = \begin{bmatrix} b \\ w \end{bmatrix}$$

$$\theta = \begin{bmatrix} b \\ w \end{bmatrix} \rightarrow \theta^T = [b \quad w]$$

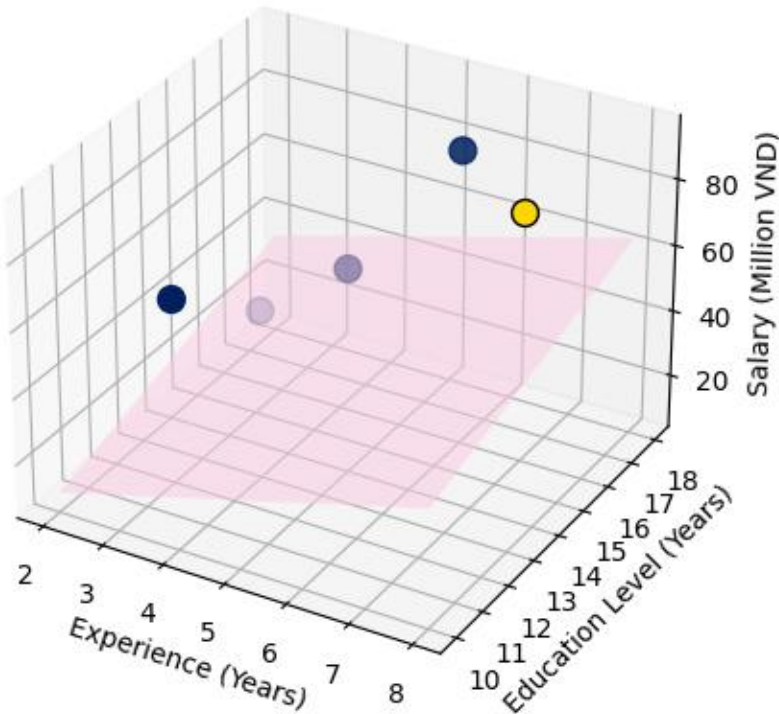
$$\hat{y} = wx + b1 = [b \quad w] \begin{bmatrix} 1 \\ x \end{bmatrix} = \theta^T x$$

dot product

Vectorized Linear Regression

❖ Example in multivariate problem

Linear Regression in 3D space



$$\hat{y} = b + w_1 x_1 + w_2 x_2$$

$$\mathbf{x} = \begin{bmatrix} 1 \\ x_1 \\ x_2 \end{bmatrix} \quad \boldsymbol{\theta} = \begin{bmatrix} b \\ w_1 \\ w_2 \end{bmatrix}$$

$$\mathbf{x} = \begin{bmatrix} 1 \\ x_1 \\ x_2 \end{bmatrix} \rightarrow \mathbf{x}^T = [1 \quad x_1 \quad x_2]$$

$$\hat{y} = b + w_1 x_1 + w_2 x_2 = [1 \quad x_1 \quad x_2] \begin{bmatrix} b \\ w_1 \\ w_2 \end{bmatrix} = \mathbf{x}^T \boldsymbol{\theta}$$

dot product

Vectorized Linear Regression

❖ Vectorized Linear Regression: Conception

1) Pick a sample (x, y) from training data

2) Compute the output $\hat{y}$

$$\hat{y} = wx + b$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\frac{\partial L}{\partial w} = 2x(\hat{y} - y)$$

$$\frac{\partial L}{\partial b} = 2(\hat{y} - y)$$

5) Update parameters

$$w = w - \eta \frac{\partial L}{\partial w}$$

$$b = b - \eta \frac{\partial L}{\partial b}$$

η is learning rate

$$\hat{y} = wx + b \quad x = \begin{bmatrix} 1 \\ x \end{bmatrix} \quad \theta = \begin{bmatrix} b \\ w \end{bmatrix}$$

$$\hat{y} = \theta^T x$$

$$L(\hat{y}, y) = (\hat{y} - y)^2$$

numbers

Vectorized Linear Regression

❖ Vectorization idea

$$\hat{y} = wx + b \quad x = \begin{bmatrix} 1 \\ x \end{bmatrix} \quad \theta = \begin{bmatrix} b \\ w \end{bmatrix}$$

$$\left\{ \begin{array}{l} \frac{\partial L}{\partial b} = 2(\hat{y} - y) = 2 \times (\hat{y} - y) \times 1 \\ \frac{\partial L}{\partial w} = 2x(\hat{y} - y) = 2 \times (\hat{y} - y) \times x \end{array} \right.$$

1) Pick a sample (x, y) from training data

2) Compute the output $\hat{y}$

$$\hat{y} = wx + b$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\frac{\partial L}{\partial w} = 2x(\hat{y} - y) \quad \frac{\partial L}{\partial b} = 2(\hat{y} - y)$$

5) Update parameters

$$w = w - \eta \frac{\partial L}{\partial w} \quad b = b - \eta \frac{\partial L}{\partial b}$$

η is learning rate

$$\begin{bmatrix} 2 \times (\hat{y} - y) \times 1 \\ 2 \times (\hat{y} - y) \times x \end{bmatrix} = \underbrace{2(\hat{y} - y)}_{\text{common factor}} \begin{bmatrix} 1 \\ x \end{bmatrix} = 2(\hat{y} - y)x = \begin{bmatrix} \frac{\partial L}{\partial b} \\ \frac{\partial L}{\partial w} \end{bmatrix} = L'_{\theta} \quad \rightarrow \quad L'_{\theta} = 2x(\hat{y} - y)$$

Vectorized Linear Regression

❖ Vectorized Linear Regression: Conception

1) Pick a sample (x, y) from training data

2) Compute the output $\hat{y}$

$$\hat{y} = wx + b$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\frac{\partial L}{\partial w} = 2x(\hat{y} - y) \quad \frac{\partial L}{\partial b} = 2(\hat{y} - y)$$

5) Update parameters

$$w = w - \eta \frac{\partial L}{\partial w} \quad b = b - \eta \frac{\partial L}{\partial b}$$

$$\begin{bmatrix} b \\ w \\ \theta \end{bmatrix} = \begin{bmatrix} b \\ w \\ \theta \end{bmatrix} - \eta \begin{bmatrix} \frac{\partial L}{\partial b} \\ \frac{\partial L}{\partial w} \\ L'_\theta \end{bmatrix}$$

$$\begin{aligned} \hat{y} &= wx + b \\ x &= \begin{bmatrix} 1 \\ x \end{bmatrix} \\ \theta &= \begin{bmatrix} b \\ w \end{bmatrix} \end{aligned} \quad L'_\theta = \begin{bmatrix} \frac{\partial L}{\partial b} \\ \frac{\partial L}{\partial w} \end{bmatrix}$$

$$\rightarrow \theta = \theta - \eta L'_\theta$$

Vectorized Linear Regression

❖ Comparison

1) Pick a sample (x, y) from training data

2) Compute the output $\hat{y}$

$$\hat{y} = wx + b$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\frac{\partial L}{\partial w} = 2x(\hat{y} - y)$$

$$\frac{\partial L}{\partial b} = 2(\hat{y} - y)$$

5) Update parameters

$$w = w - \eta \frac{\partial L}{\partial w}$$

$$b = b - \eta \frac{\partial L}{\partial b}$$

η is learning rate

Simple

1) Pick a sample (x, y) from training data

2) Compute output $\hat{y}$

$$\hat{y} = \theta^T x = x^T \theta$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$L'_\theta = 2x(\hat{y} - y)$$

5) Update parameters

$$\theta = \theta - \eta L'_\theta$$

η is learning rate

Vectorized

Vectorized Linear Regression

❖ N-samples vectorization: Vectorize all samples

Experience	Education	Salary
3	12	60
4	13	55
5	14	66
6	15	93
7	16	?

$$\begin{aligned} \mathbf{x}^{(1)} &= \begin{bmatrix} 1 \\ x_1^{(1)} \\ x_2^{(1)} \end{bmatrix} \\ \mathbf{x}^{(2)} &= \begin{bmatrix} 1 \\ x_1^{(2)} \\ x_2^{(2)} \end{bmatrix} \end{aligned} \quad \left. \vphantom{\begin{bmatrix} 1 \\ x_1^{(1)} \\ x_2^{(1)} \end{bmatrix}} \right\} \quad \mathbf{x} = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} \end{bmatrix}$$

Vectorized Linear Regression

❖ N-samples vectorization: Prediction

$$\hat{y}^{(1)} = b + w_1 x_1^{(1)} + w_2 x_2^{(1)}$$

$$\hat{y}^{(2)} = b + w_1 x_1^{(2)} + w_2 x_2^{(2)}$$

Experience	Education	Salary
3	12	60
4	13	55
5	14	66
6	15	93
7	16	?

$$\mathbf{x} = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} \end{bmatrix} \quad \boldsymbol{\theta} = \begin{bmatrix} b \\ w_1 \\ w_2 \end{bmatrix}$$

$$\hat{\mathbf{y}} = \mathbf{x}\boldsymbol{\theta} = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} \end{bmatrix} \begin{bmatrix} b \\ w_1 \\ w_2 \end{bmatrix} = \begin{bmatrix} b + w_1 x_1^{(1)} + w_2 x_2^{(1)} \\ b + w_1 x_1^{(2)} + w_2 x_2^{(2)} \end{bmatrix} = \begin{bmatrix} \hat{y}^{(1)} \\ \hat{y}^{(2)} \end{bmatrix}$$

Vectorized Linear Regression

❖ N-samples vectorization: Loss

Experience	Education	Salary
3	12	60
4	13	55
5	14	66
6	15	93
7	16	?

Compute loss

$$L = (\hat{y} - y)^2$$

$$\hat{\mathbf{y}} = \begin{bmatrix} \hat{y}^{(1)} \\ \hat{y}^{(2)} \end{bmatrix} = \mathbf{x}\boldsymbol{\theta}$$

$$L(\hat{\mathbf{y}}, \mathbf{y}) = \begin{bmatrix} L(\hat{y}^{(1)}, y^{(1)}) \\ L(\hat{y}^{(2)}, y^{(2)}) \end{bmatrix} = \begin{bmatrix} (\hat{y}^{(1)} - y^{(1)})^2 \\ (\hat{y}^{(2)} - y^{(2)})^2 \end{bmatrix} = \begin{bmatrix} (\hat{y}^{(1)} - y^{(1)}) * (\hat{y}^{(1)} - y^{(1)}) \\ (\hat{y}^{(2)} - y^{(2)}) * (\hat{y}^{(2)} - y^{(2)}) \end{bmatrix}$$

$$= \begin{bmatrix} (\hat{y}^{(1)} - y^{(1)}) \\ (\hat{y}^{(2)} - y^{(2)}) \end{bmatrix} \odot \begin{bmatrix} (\hat{y}^{(1)} - y^{(1)}) \\ (\hat{y}^{(2)} - y^{(2)}) \end{bmatrix} = (\hat{\mathbf{y}} - \mathbf{y}) \odot (\hat{\mathbf{y}} - \mathbf{y})$$

Hadamard product

Vectorized Linear Regression

❖ N-samples vectorization: Derivative

propagate and accumulate

Experience	Education	Salary
3	12	60
4	13	55
5	14	66
6	15	93
7	16	?

two samples

$$k = 2(\hat{\mathbf{y}} - \mathbf{y}) = \begin{bmatrix} 2(\hat{y}^{(1)} - y^{(1)}) \\ 2(\hat{y}^{(2)} - y^{(2)}) \end{bmatrix}$$

$$\mathbf{x} = \begin{bmatrix} 1 & 6.7 \\ 1 & 4.6 \end{bmatrix}$$

way 1

$$\begin{bmatrix} 2(\hat{y}^{(1)} - y^{(1)}) \\ 2(\hat{y}^{(2)} - y^{(2)}) \end{bmatrix} \begin{bmatrix} 1 & 6.7 \\ 1 & 4.6 \end{bmatrix} = \begin{bmatrix} \frac{\partial L}{\partial b} & \frac{\partial L}{\partial w} \end{bmatrix} = \mathbf{k}^T \mathbf{x}$$

way 2

$$\begin{bmatrix} 1 & 1 \\ 6.7 & 4.6 \end{bmatrix} \begin{bmatrix} 2(\hat{y}^{(1)} - y^{(1)}) \\ 2(\hat{y}^{(2)} - y^{(2)}) \end{bmatrix} = \begin{bmatrix} \frac{\partial L}{\partial b} \\ \frac{\partial L}{\partial w} \end{bmatrix} = \mathbf{x}^T \mathbf{k}$$

Vectorized Linear Regression

❖ N-samples vectorization: Loss

Experience	Education	Salary
3	12	60
4	13	55
5	14	66
6	15	93
7	16	?

Update parameters

$$w = w - \eta \frac{\partial L}{\partial w}$$

$$b = b - \eta \frac{\partial L}{\partial b}$$

$$\left\{ \begin{array}{l} b \\ w \\ \theta \end{array} \right\} = \left\{ \begin{array}{l} b \\ w \\ \theta \end{array} \right\} - \eta \left\{ \begin{array}{l} \frac{\partial L}{\partial b} \\ \frac{\partial L}{\partial w} \\ L'_\theta \end{array} \right\}$$

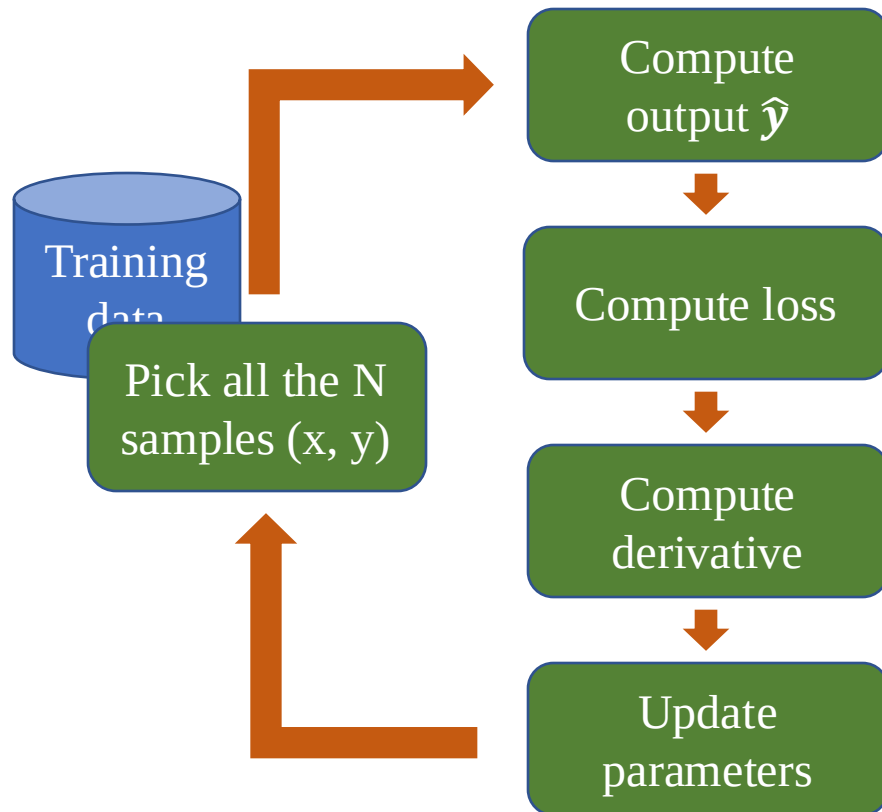
$$L'_\theta = \begin{bmatrix} \frac{\partial L}{\partial b} \\ \frac{\partial L}{\partial w} \end{bmatrix} = \mathbf{x}^T \mathbf{k}$$

$$L'_\theta = \begin{bmatrix} \frac{\partial L}{\partial b} & \frac{\partial L}{\partial w} \end{bmatrix} = \mathbf{k}^T \mathbf{x}$$

$$\rightarrow \boldsymbol{\theta} = \boldsymbol{\theta} - \eta L'_\theta$$

Vectorized Linear Regression

❖ N-samples vectorization



1) Pick all the N samples from training data

2) Compute output $\hat{y}$

$$\hat{y} = x\theta$$

3) Compute loss

$$L(\hat{y}, y) = \frac{1}{N} (\hat{y} - y)^T \odot (\hat{y} - y)$$

4) Compute derivative

$$k = 2(\hat{y} - y) \quad L'_{\theta} = x^T k$$

5) Update parameters

$$\theta = \theta - \eta \frac{L'_{\theta}}{N} \quad \eta \text{ is learning rate}$$

Vectorized Linear Regression

❖ Forward # 1

Initialize

$$w_1 = 3$$

$$w_2 = 2$$

$$b = 10$$

$$\eta = 0.001$$

$$N = 4$$

$$N_EPOCH = 2$$

Input Vector

$$x = \begin{bmatrix} 1 & 3 & 12 \\ 1 & 4 & 13 \\ 1 & 5 & 14 \\ 1 & 6 & 15 \end{bmatrix}$$

Model

$$\begin{matrix} w_1 = 3 \\ w_2 = 2 \end{matrix}$$

$$b = 10$$

$$\theta = \begin{bmatrix} b \\ w_1 \\ w_2 \end{bmatrix}$$

$$\hat{y} = x\theta = \begin{bmatrix} 43 \\ 48 \\ 53 \\ 58 \end{bmatrix}$$

Experience	Education	Salary
3	12	60
4	13	55
5	14	66
6	15	93

Label Vector

$$y = \begin{bmatrix} 60 \\ 55 \\ 66 \\ 93 \end{bmatrix}$$

Epoch: 1

$$L(\hat{y}, y) = \frac{1}{N} (\hat{y} - y)^T \odot (\hat{y} - y) = 433$$

Loss

Vectorized Linear Regression

❖ Backpropagation #1

Initialize

$$w_1 = 3$$

$$w_2 = 2$$

$$b = 10$$

$$\eta = 0.001$$

$$N = 4$$

$$N_EPOCH = 2$$

Input Vector

$$x = \begin{bmatrix} 1 & 3 & 12 \\ 1 & 4 & 13 \\ 1 & 5 & 14 \\ 1 & 6 & 15 \end{bmatrix}$$

Model

$$\begin{matrix} w_1 = 3.177 \\ w_2 = 2.501 \end{matrix}$$

$$b = 10.036$$

$$\theta = \begin{bmatrix} b \\ w_1 \\ w_2 \end{bmatrix}$$

$$\hat{y} = x\theta = \begin{bmatrix} 43 \\ 48 \\ 53 \\ 58 \end{bmatrix}$$

Label Vector

$$y = \begin{bmatrix} 60 \\ 55 \\ 66 \\ 93 \end{bmatrix}$$

$$L(\hat{y}, y) = \frac{1}{N} (\hat{y} - y)^T \odot (\hat{y} - y) = 433$$

Loss

Vectorized Linear Regression

❖ Forward # 2

Initialize

$$w_1 = 3$$

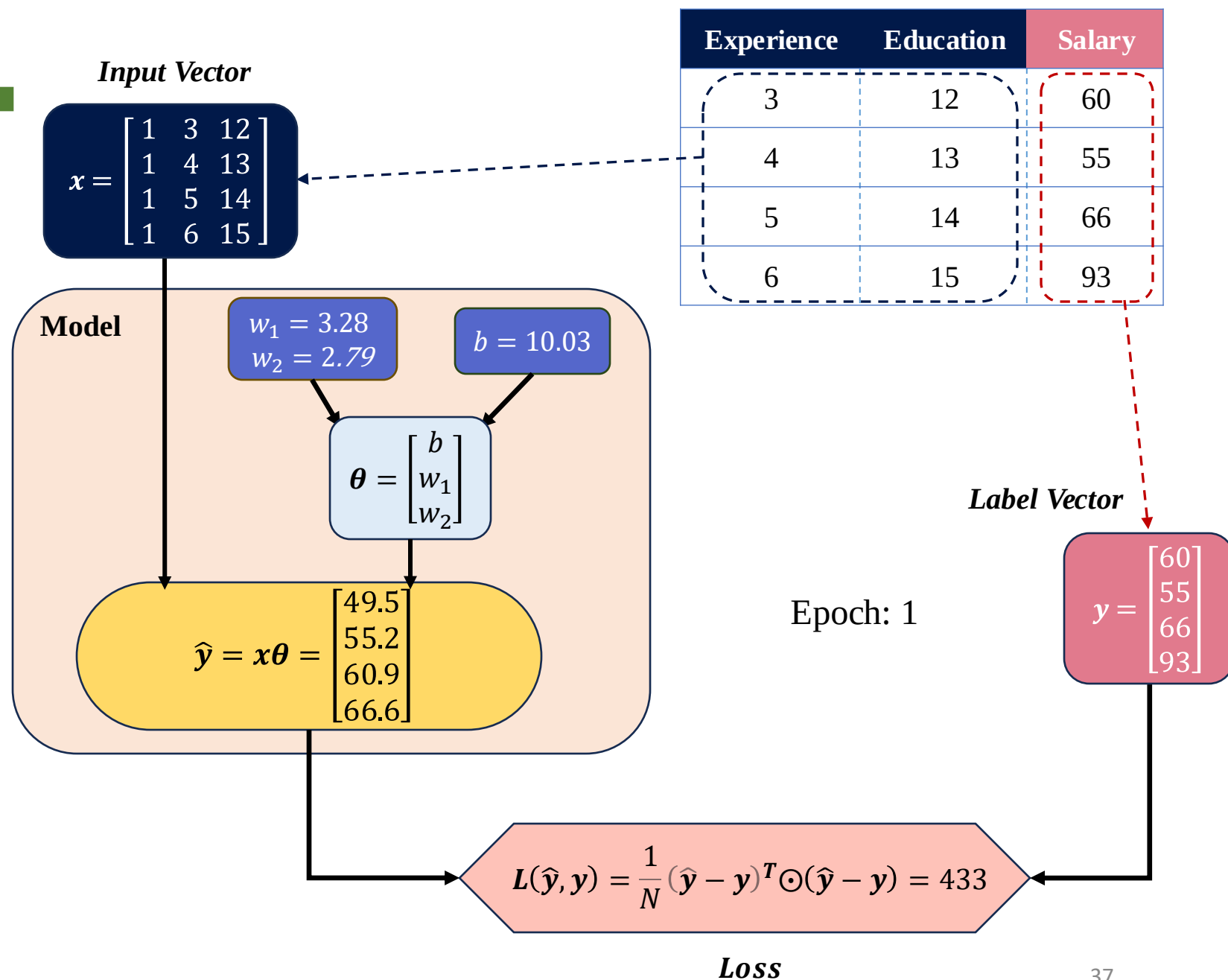
$$w_2 = 2$$

$$b = 10$$

$$\eta = 0.001$$

$$N = 4$$

$$N_EPOCH = 2$$



Vectorized Linear Regression

❖ Backpropagation #2

Initialize

$$w_1 = 3$$

$$w_2 = 2$$

$$b = 10$$

$$\eta = 0.001$$

$$N = 4$$

$$N_EPOCH = 2$$

Input Vector

$$x = \begin{bmatrix} 1 & 3 & 12 \\ 1 & 4 & 13 \\ 1 & 5 & 14 \\ 1 & 6 & 15 \end{bmatrix}$$

Model

$$\begin{matrix} w_1 = 3.283 \\ w_2 = 2.795 \end{matrix}$$

$$b = 10.056$$

$$\theta = \begin{bmatrix} b \\ w_1 \\ w_2 \end{bmatrix}$$

$$\hat{y} = x\theta = \begin{bmatrix} 49.5 \\ 55.2 \\ 60.9 \\ 66.6 \end{bmatrix}$$

Label Vector

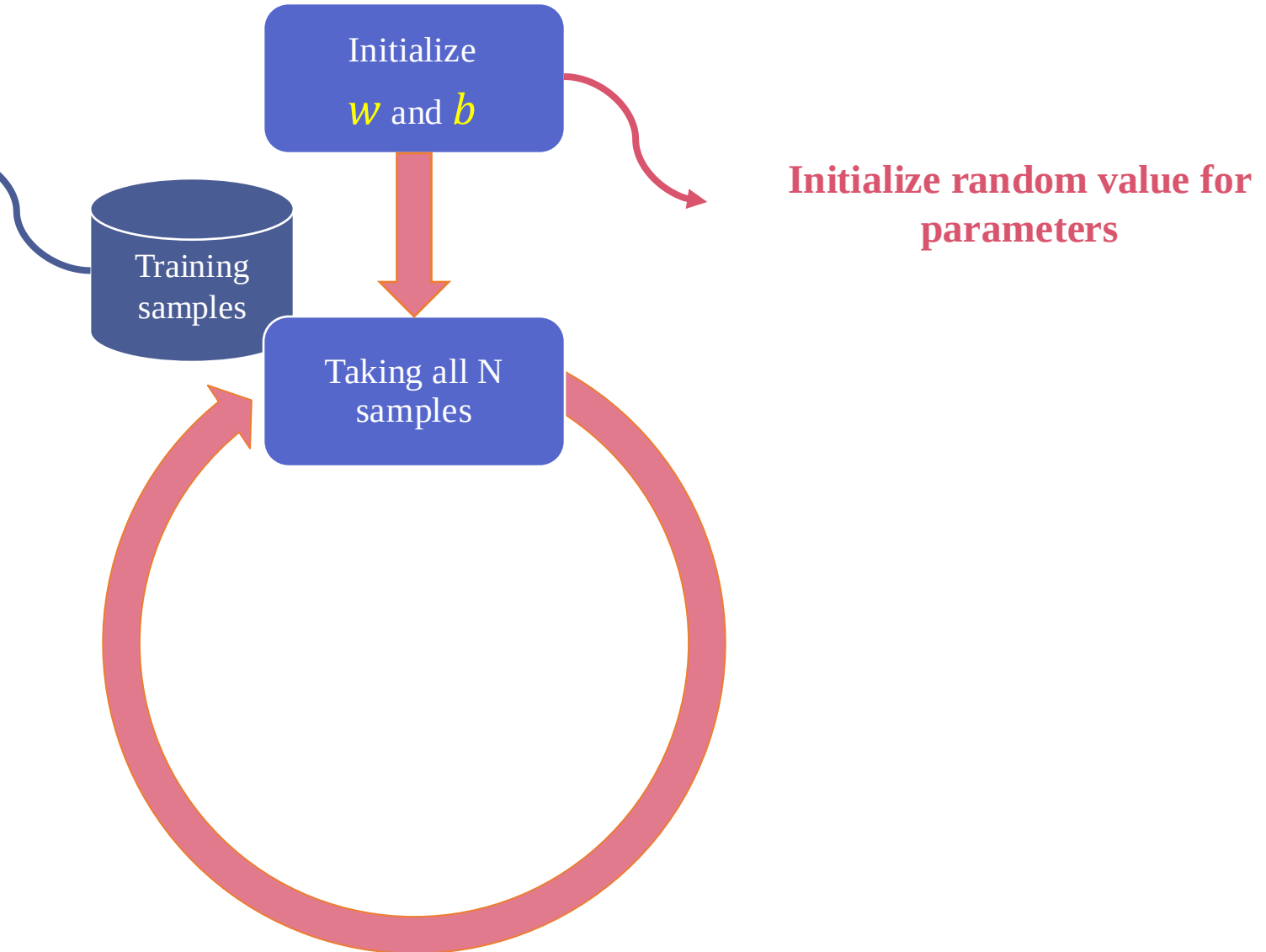
$$y = \begin{bmatrix} 60 \\ 55 \\ 66 \\ 93 \end{bmatrix}$$

$$L(\hat{y}, y) = \frac{1}{N} (\hat{y} - y)^T \odot (\hat{y} - y) = 433$$

Loss

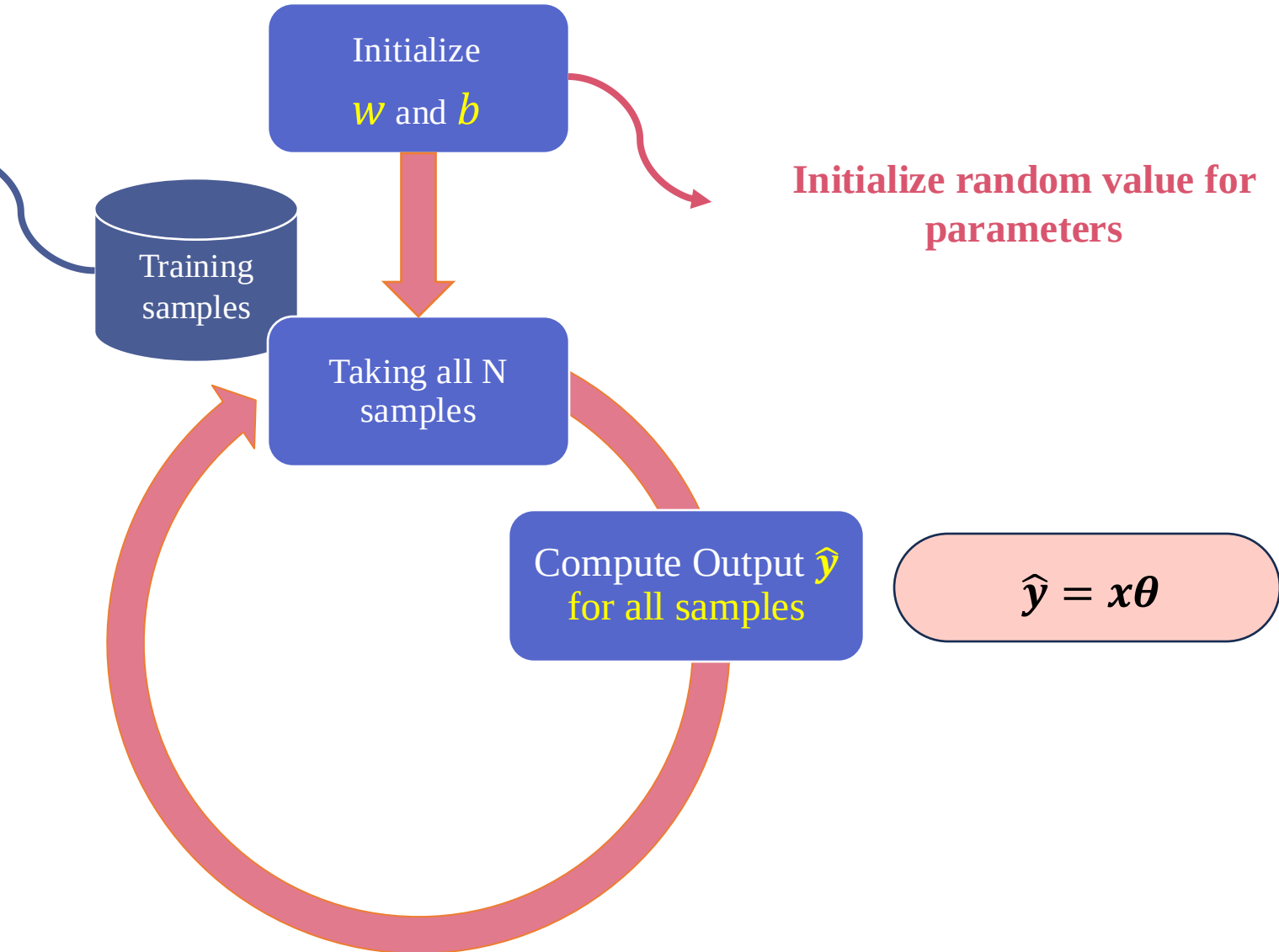
Vectorized Linear Regression

	Experience	Education	Salary
0	3	12	60
1	4	13	55
2	5	14	66
3	6	15	93



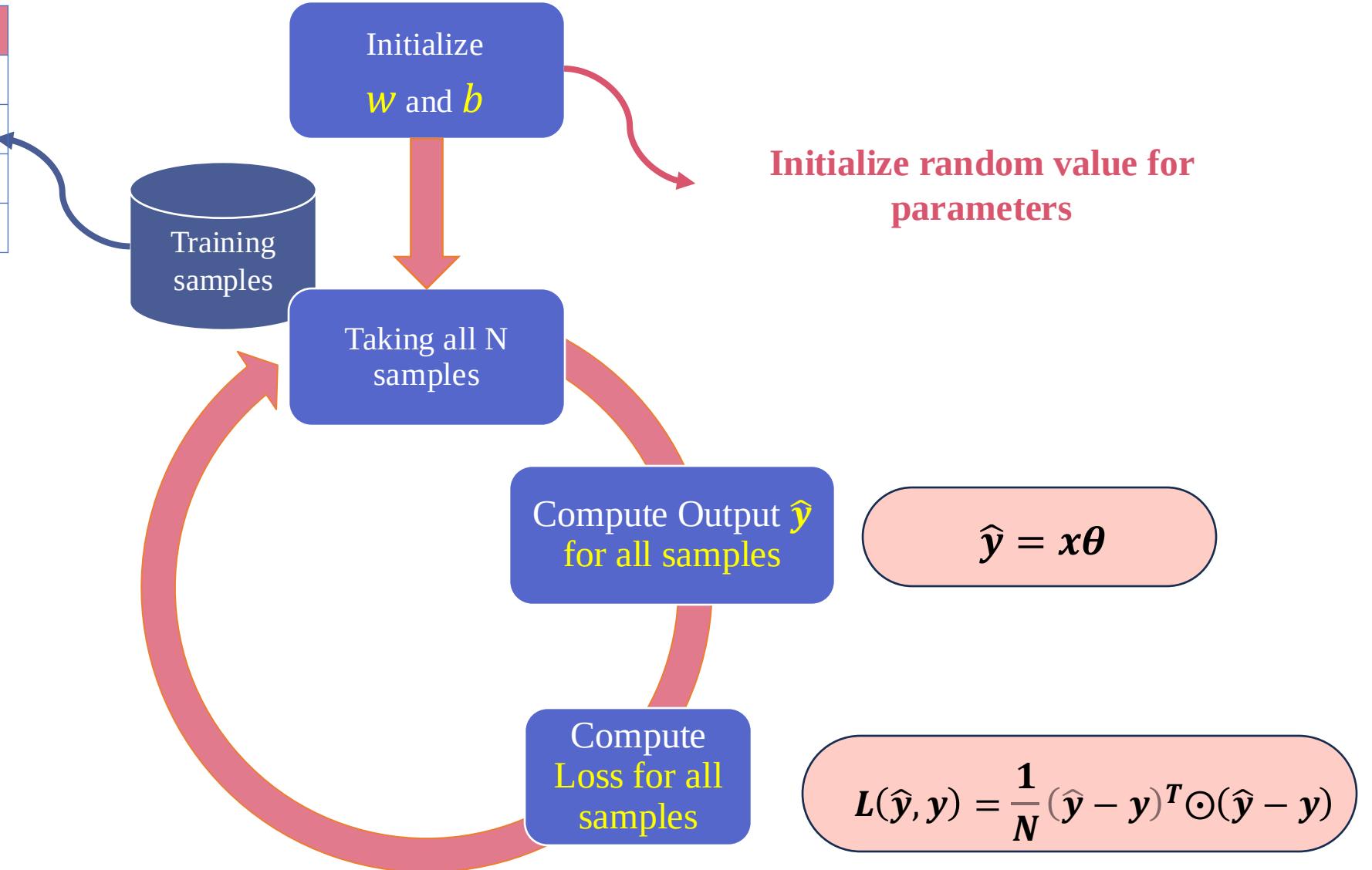
Vectorized Linear Regression

	Experience	Education	Salary
0	3	12	60
1	4	13	55
2	5	14	66
3	6	15	93



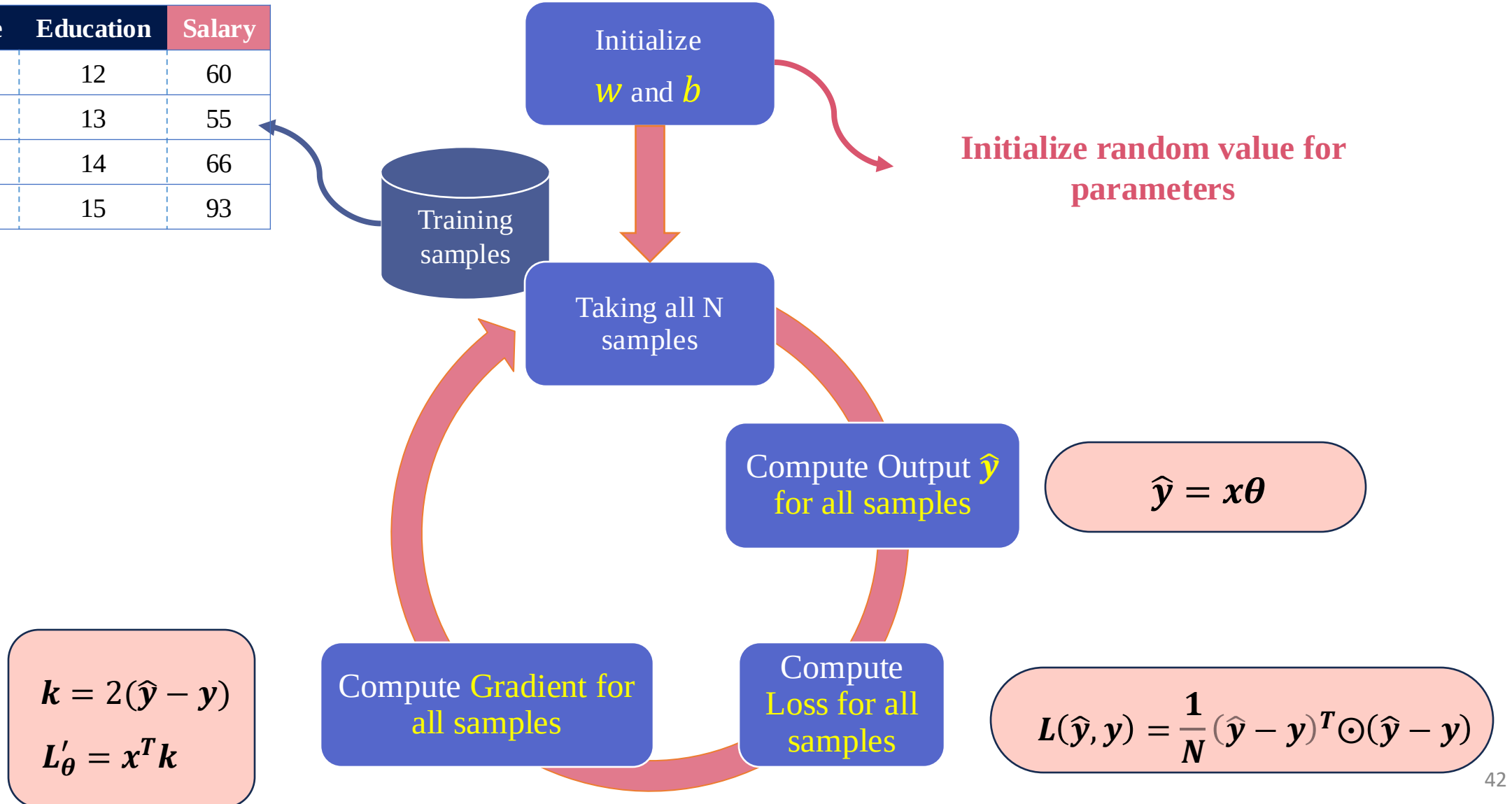
Vectorized Linear Regression

	Experience	Education	Salary
0	3	12	60
1	4	13	55
2	5	14	66
3	6	15	93



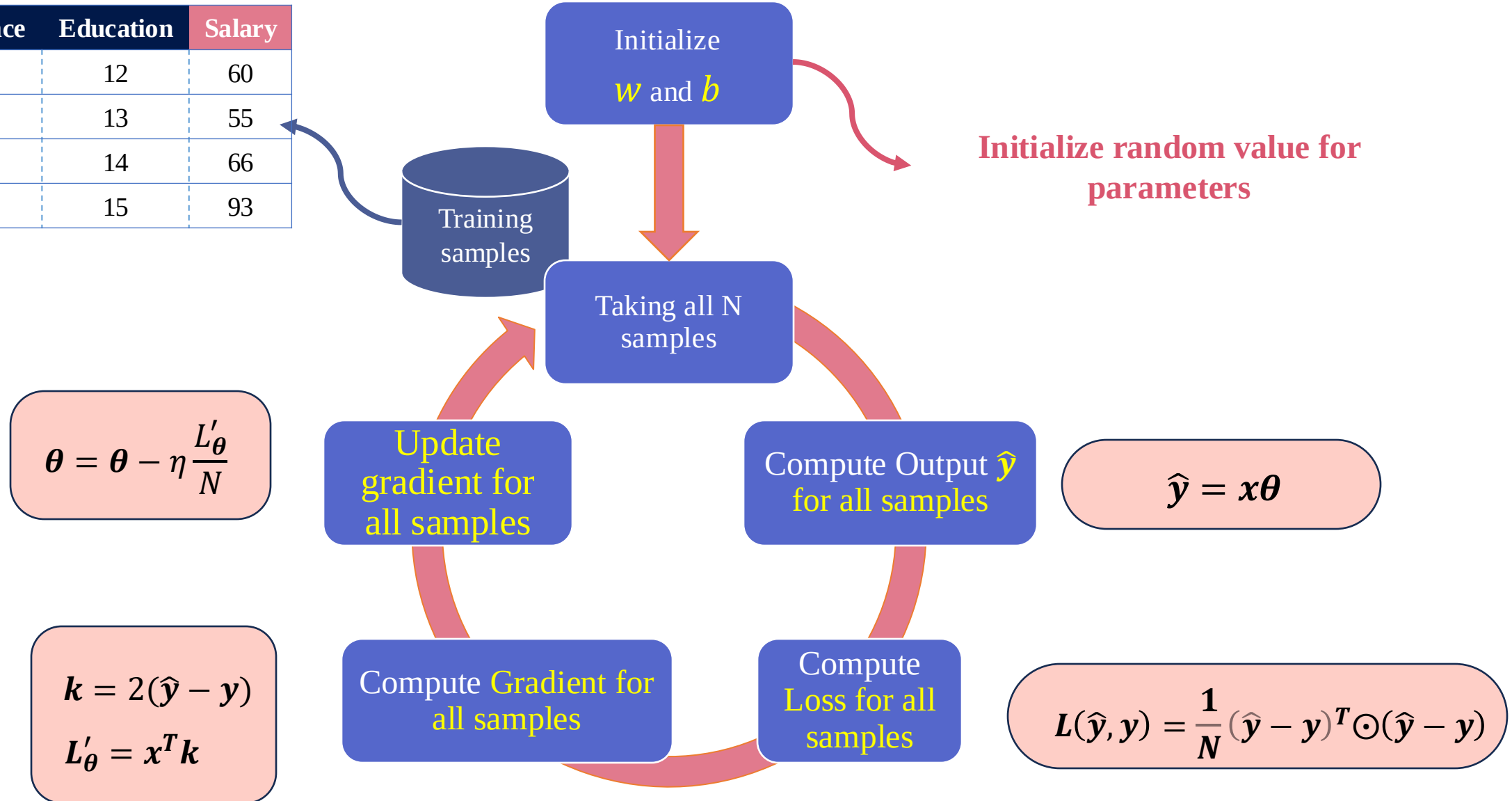
Vectorized Linear Regression

	Experience	Education	Salary
0	3	12	60
1	4	13	55
2	5	14	66
3	6	15	93



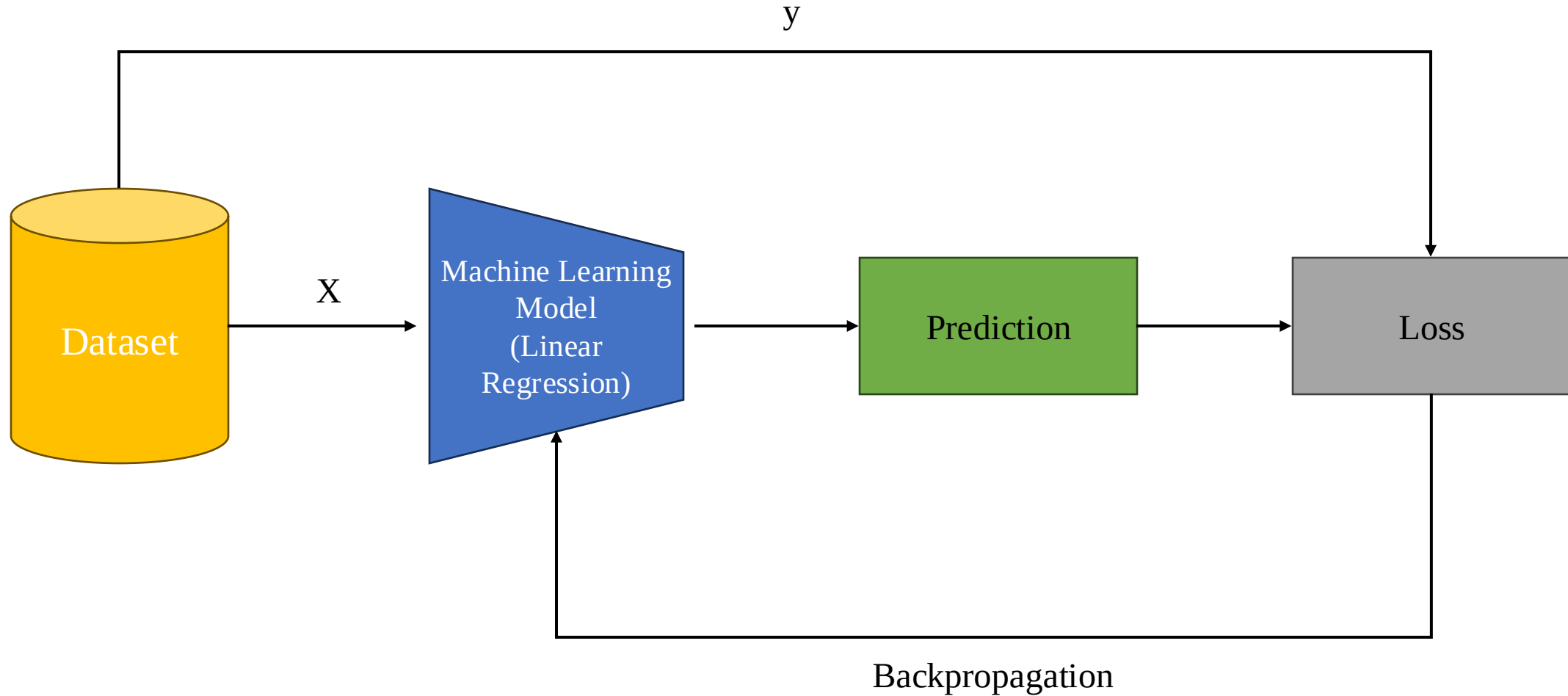
Vectorized Linear Regression

	Experience	Education	Salary
0	3	12	60
1	4	13	55
2	5	14	66
3	6	15	93



Vectorized Linear Regression

❖ Training Cycle

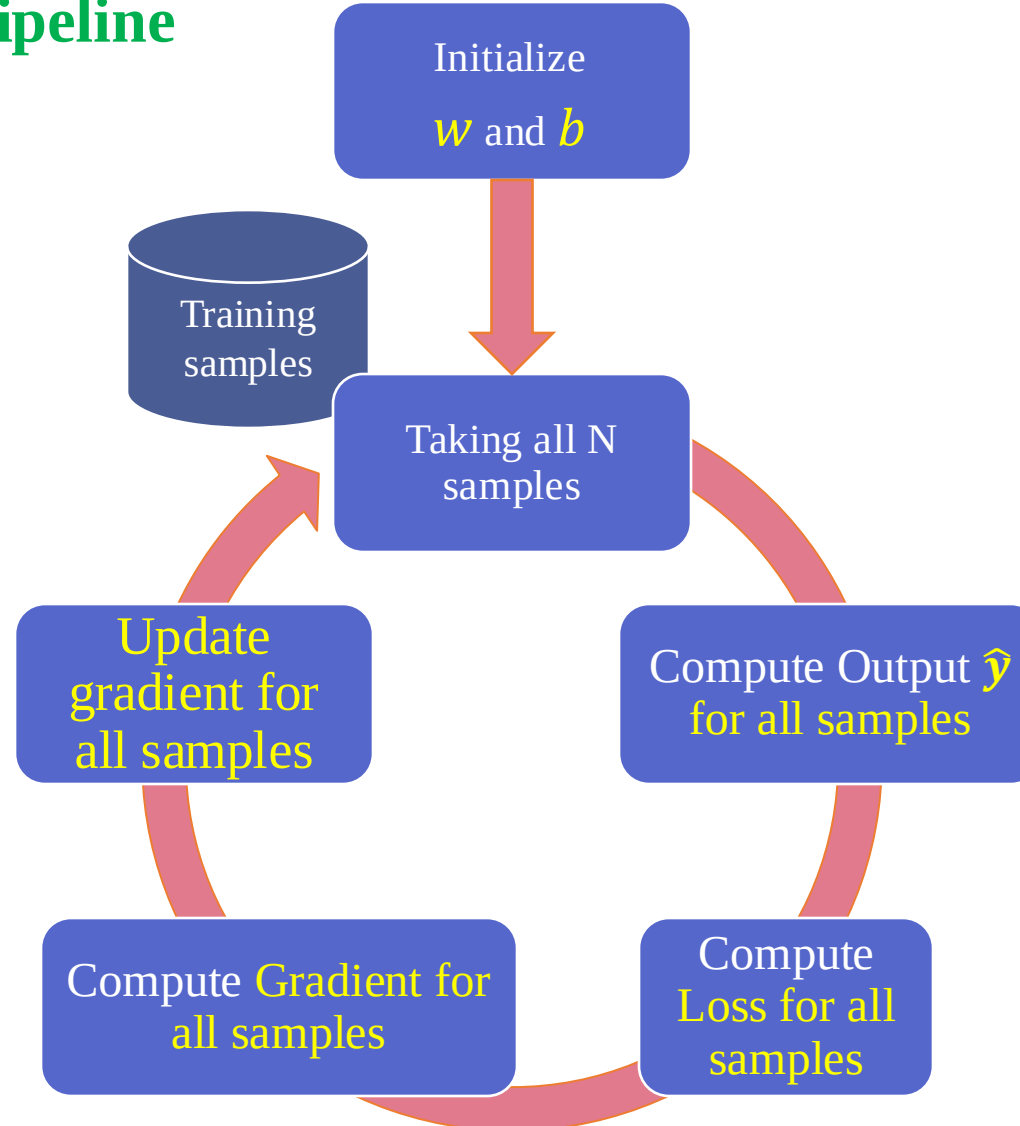


QUIZ

Code Implementation

Code Implementation

❖ Gradient Descent Pipeline



Code Implementation

❖ Versions 1: 1-sample

1) Pick a sample (x, y) from training data

2) Compute output $\hat{y}$

$$\hat{y} = \theta^T x = x^T \theta$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$L'_{\theta} = 2x(\hat{y} - y)$$

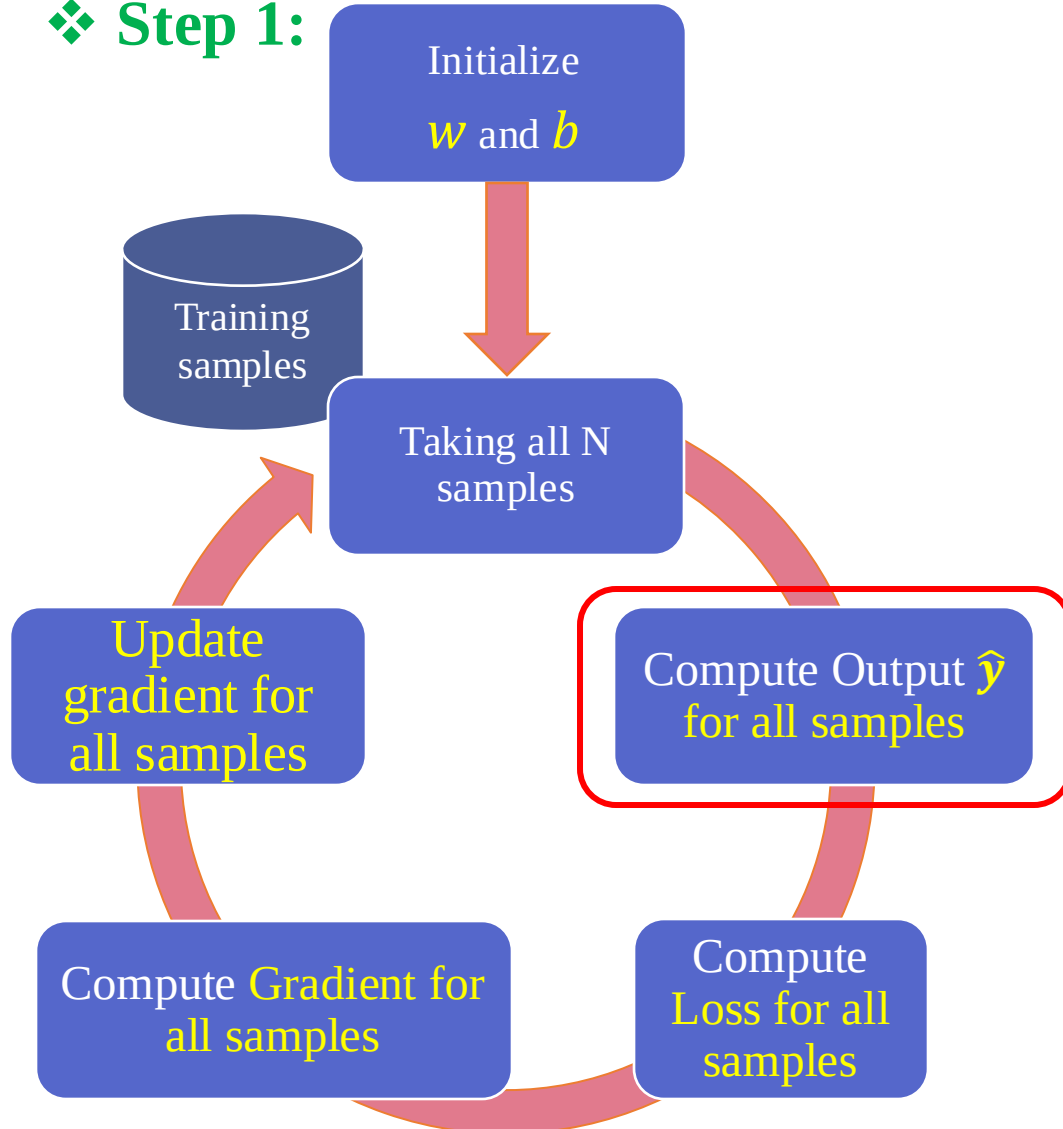
5) Update parameters

$$\theta = \theta - \eta L'_{\theta}$$

η is learning rate

Code Implementation

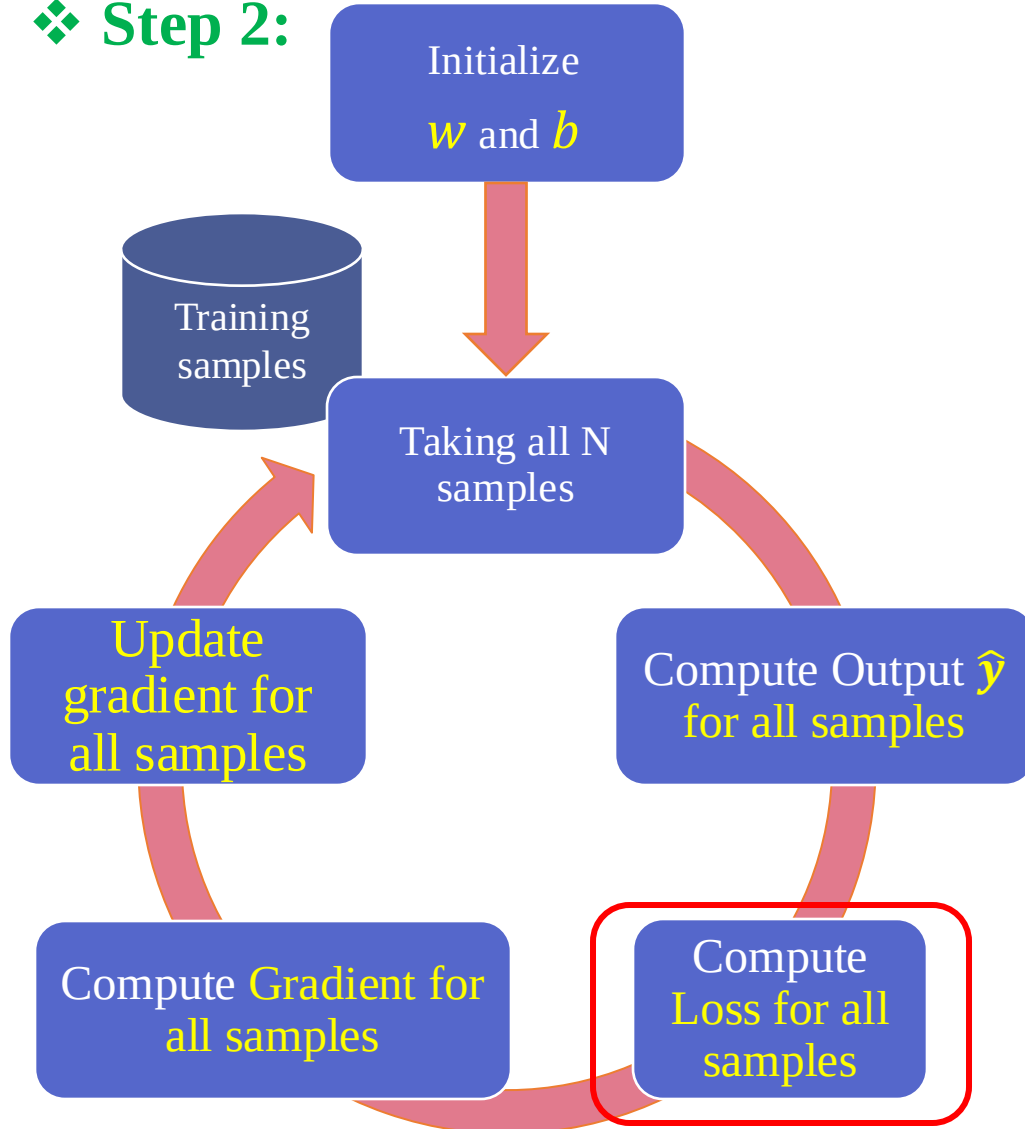
❖ Step 1:



```
1 def predict(X, theta):
2     return X.dot(theta)
3
4 def compute_loss(y_hat, y):
5
6     return (y_hat - y) ** 2
7
8 def compute_gradient(y_hat, y, X):
9
10    return 2 * X * (y_hat - y)
11
12 def update_gradient(theta, gradient, lr):
13     theta = theta - lr * gradient
14
15     return theta
```

Code Implementation

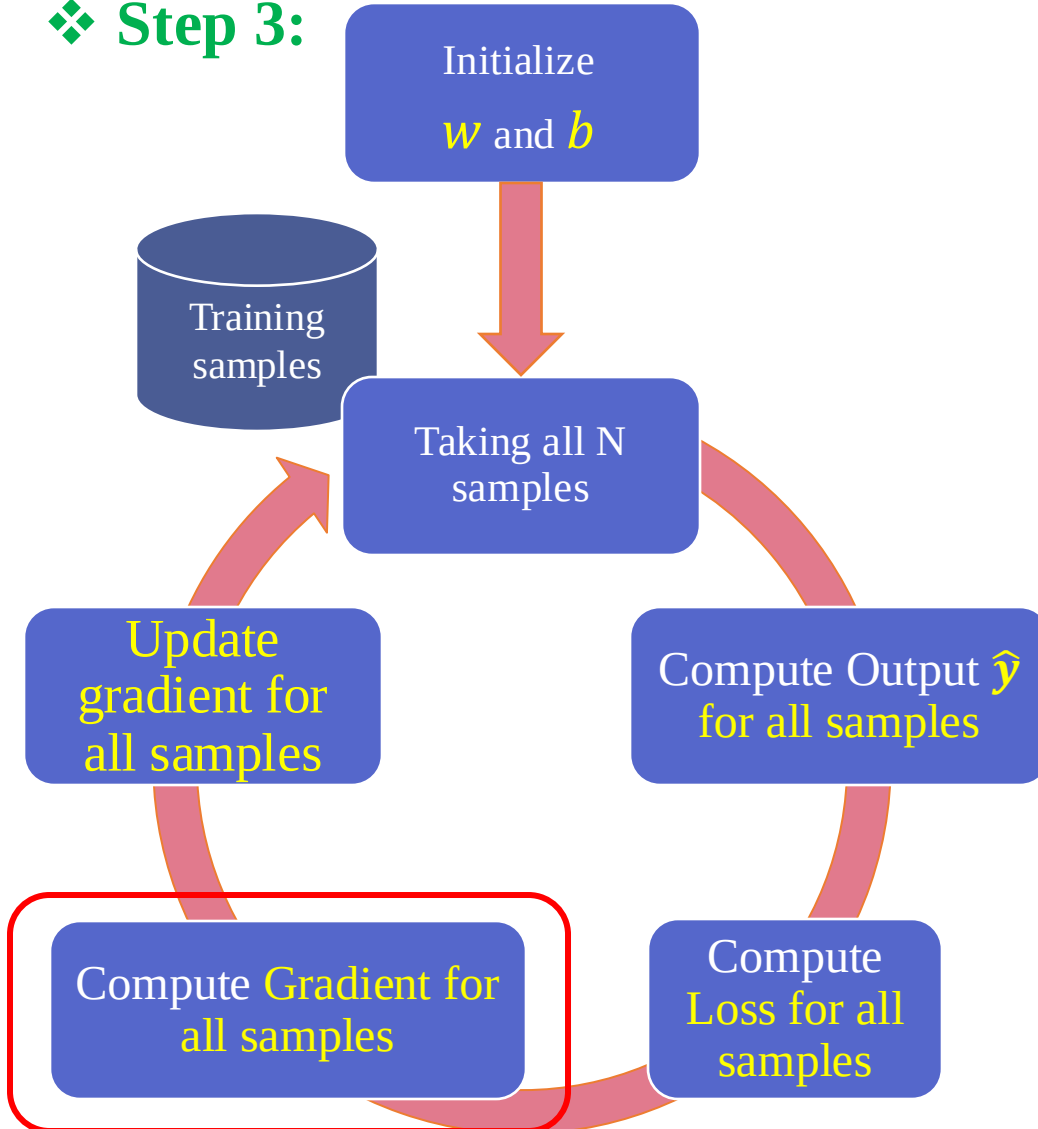
❖ Step 2:



```
1 def predict(X, theta):
2     return X.dot(theta)
3
4 def compute_loss(y_hat, y):
5
6     return (y_hat - y) ** 2
7
8 def compute_gradient(y_hat, y, X):
9
10    return 2 * X * (y_hat - y)
11
12 def update_gradient(theta, gradient, lr):
13     theta = theta - lr * gradient
14
15     return theta
```

Code Implementation

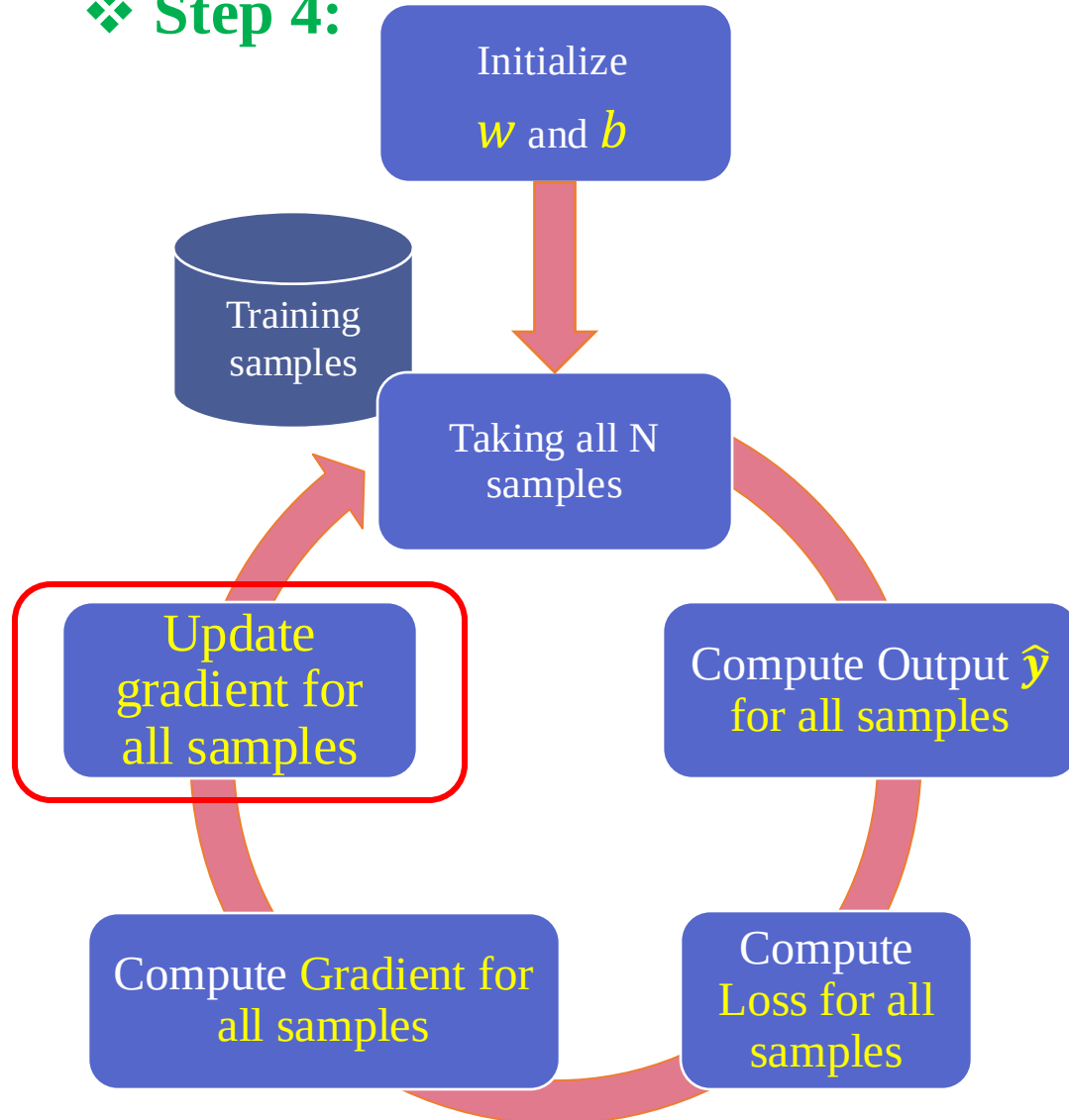
❖ Step 3:



```
1 def predict(X, theta):
2     return X.dot(theta)
3
4 def compute_loss(y_hat, y):
5
6     return (y_hat - y) ** 2
7
8 def compute_gradient(y_hat, y, X):
9
10    return 2 * X * (y_hat - y)
11
12 def update_gradient(theta, gradient, lr):
13     theta = theta - lr * gradient
14
15     return theta
```

Code Implementation

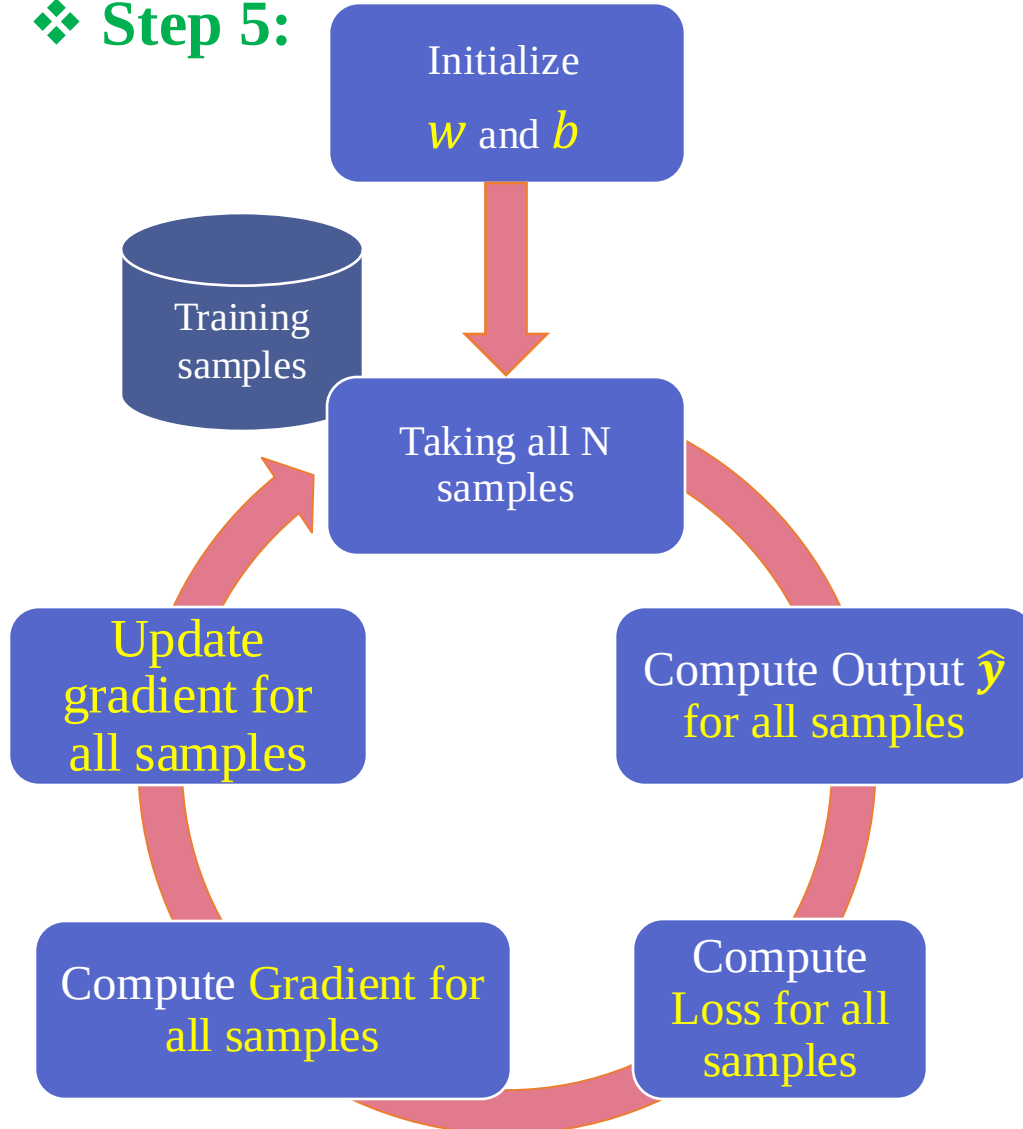
❖ Step 4:



```
1 def predict(X, theta):
2     return X.dot(theta)
3
4 def compute_loss(y_hat, y):
5
6     return (y_hat - y) ** 2
7
8 def compute_gradient(y_hat, y, X):
9
10    return 2 * X * (y_hat - y)
11
12 def update_gradient(theta, gradient, lr):
13     theta = theta - lr * gradient
14
15     return theta
```

Code Implementation

❖ Step 5:



```
1 epoch_max = 4
2 lr = 0.001 # Learning rate
3 N = len(y)
4 losses = []
5
6 for epoch in range(epoch_max):
7     epoch_losses = []
8     for idx in range(N):
9         # Lấy mẫu dữ liệu thứ idx
10        X_i = X_bias[idx, :]
11        y_i = y[idx]
12
13        # Dự đoán giá trị
14        y_hat = predict(X_i, theta)
15
16        # Tính Loss (MSE) - Chia cho N
17        loss = compute_loss(y_hat, y_i)
18        epoch_losses.append(loss)
19
20        # Tính gradient
21        gradient = compute_gradient(y_hat, y_i, X_i)
22
23        # Cập nhật tham số
24        theta = update_gradient(theta, gradient, lr)
25
26    epoch_loss = sum(epoch_losses) / len(epoch_losses)
27    losses.append(epoch_loss)
28    print(f'Epoch {epoch+1}: Loss = {epoch_loss}, theta = {theta}')
```

Code Implementation

❖ Versions 2: N-samples

1) Pick all the N samples from training data

2) Compute output $\hat{\mathbf{y}}$

$$\hat{\mathbf{y}} = \mathbf{x}\boldsymbol{\theta}$$

3) Compute loss

$$L(\hat{\mathbf{y}}, \mathbf{y}) = \frac{1}{N} (\hat{\mathbf{y}} - \mathbf{y})^T \odot (\hat{\mathbf{y}} - \mathbf{y})$$

4) Compute derivative

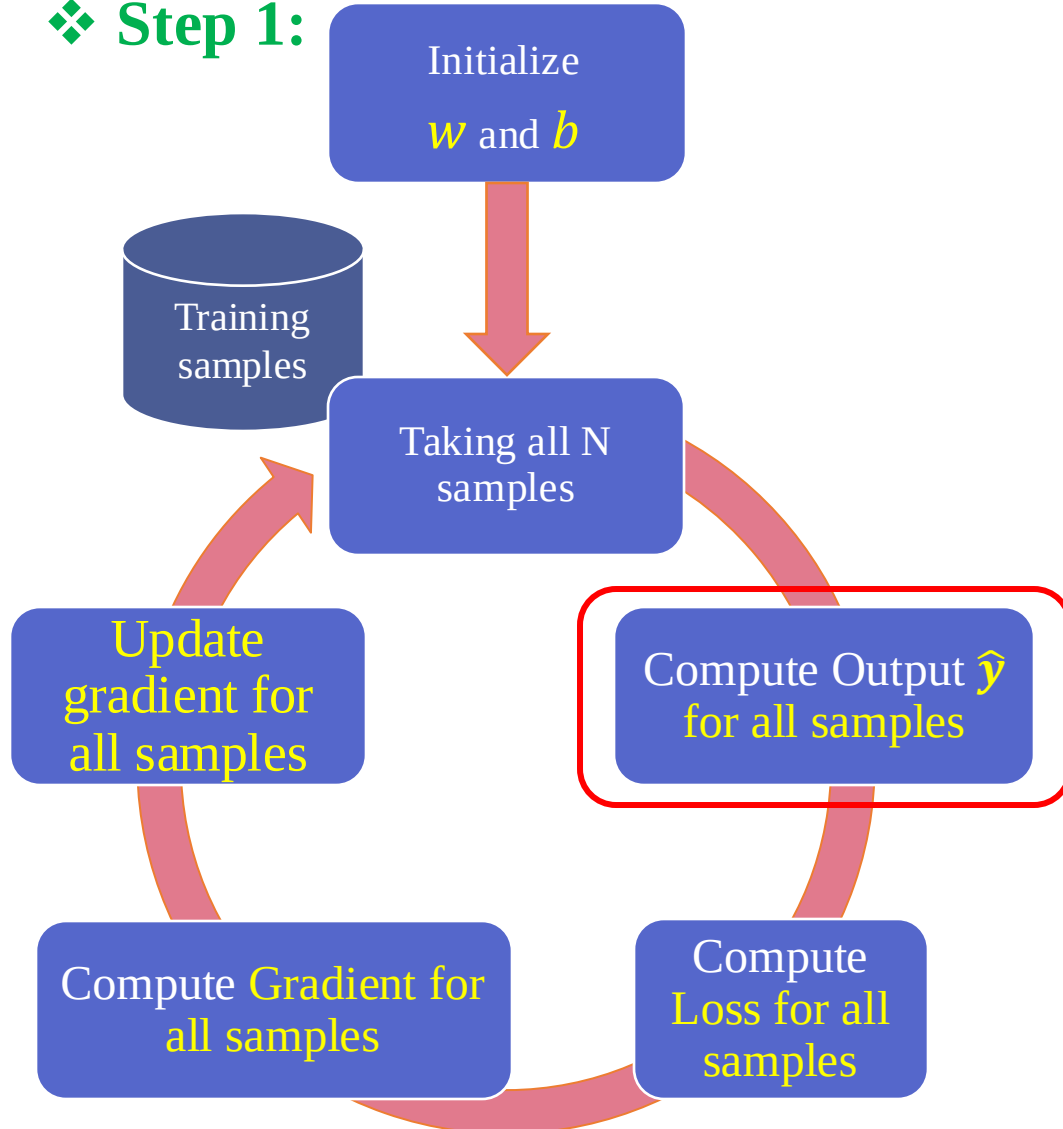
$$\mathbf{k} = 2(\hat{\mathbf{y}} - \mathbf{y}) \quad L'_{\boldsymbol{\theta}} = \mathbf{x}^T \mathbf{k}$$

5) Update parameters

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \frac{L'_{\boldsymbol{\theta}}}{N} \quad \eta \text{ is learning rate}$$

Code Implementation

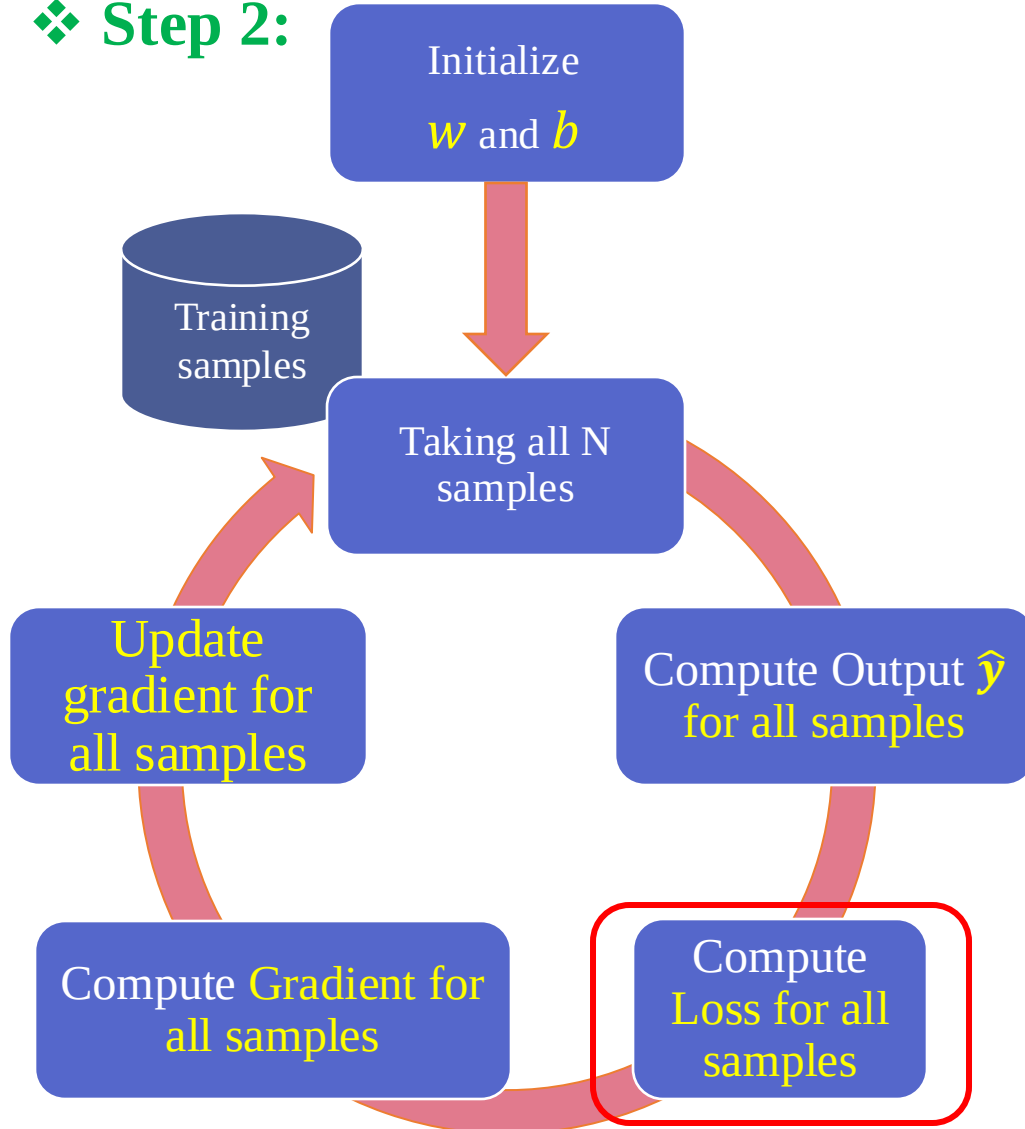
❖ Step 1:



```
1 import numpy as np
2
3 def predict(X, theta):
4     return X.dot(theta)
5
6 def compute_loss(y_hat, y):
7     N = y.shape[0]
8
9     return np.sum((y_hat - y) ** 2) / N
10
11 def compute_gradient(y_hat, y, X):
12     N = y.shape[0]
13     k = 2 * (y_hat - y)
14
15     return X.T.dot(k) / N
16
17 def update_gradient(theta, gradient, lr):
18     theta = theta - lr * gradient
19
20     return theta
```

Code Implementation

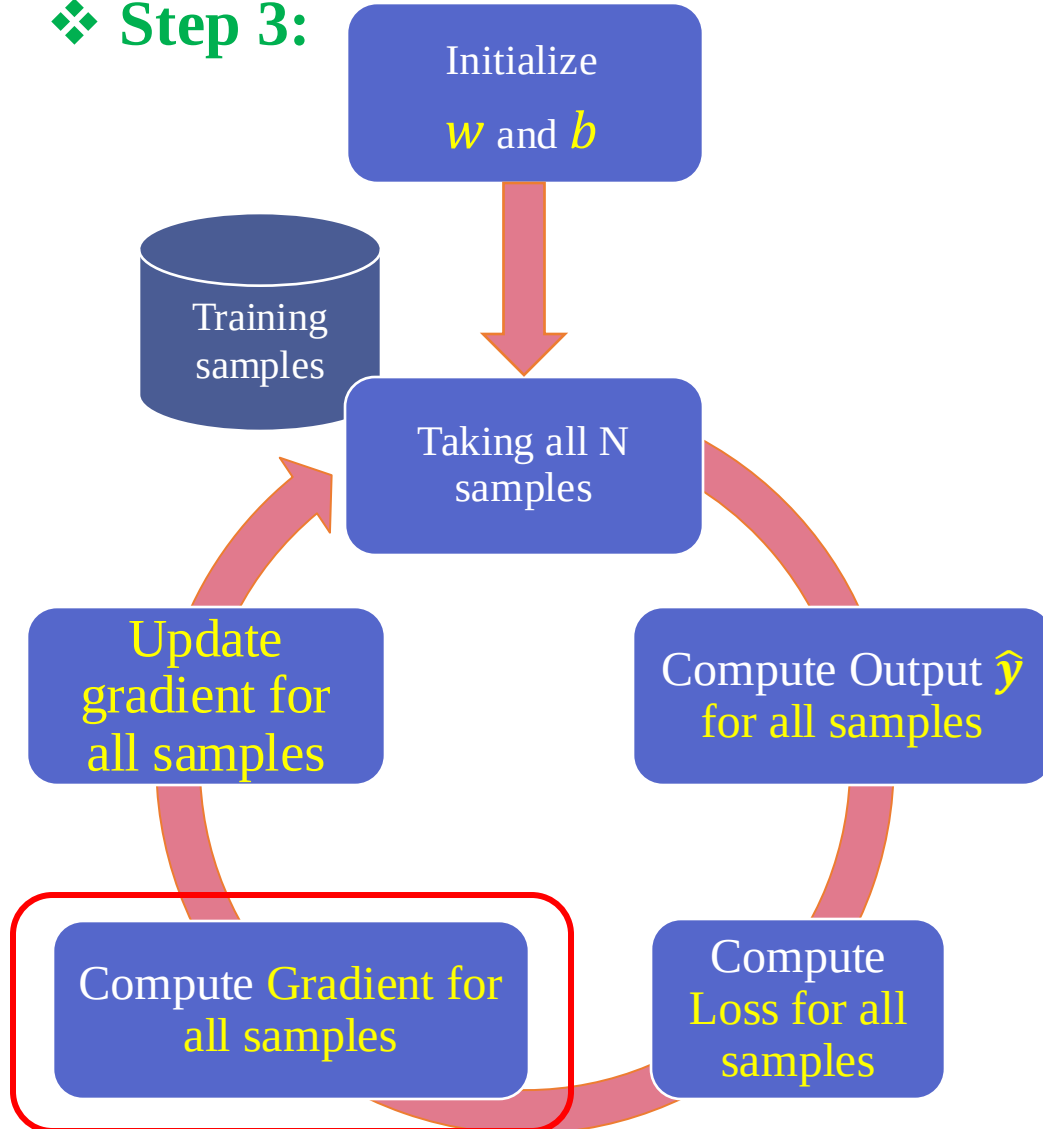
❖ Step 2:



```
1 import numpy as np
2
3 def predict(X, theta):
4     return X.dot(theta)
5
6 def compute_loss(y_hat, y):
7     N = y.shape[0]
8
9     return np.sum((y_hat - y) ** 2) / N
10
11 def compute_gradient(y_hat, y, X):
12     N = y.shape[0]
13     k = 2 * (y_hat - y)
14
15     return X.T.dot(k) / N
16
17 def update_gradient(theta, gradient, lr):
18     theta = theta - lr * gradient
19
20     return theta
```

Code Implementation

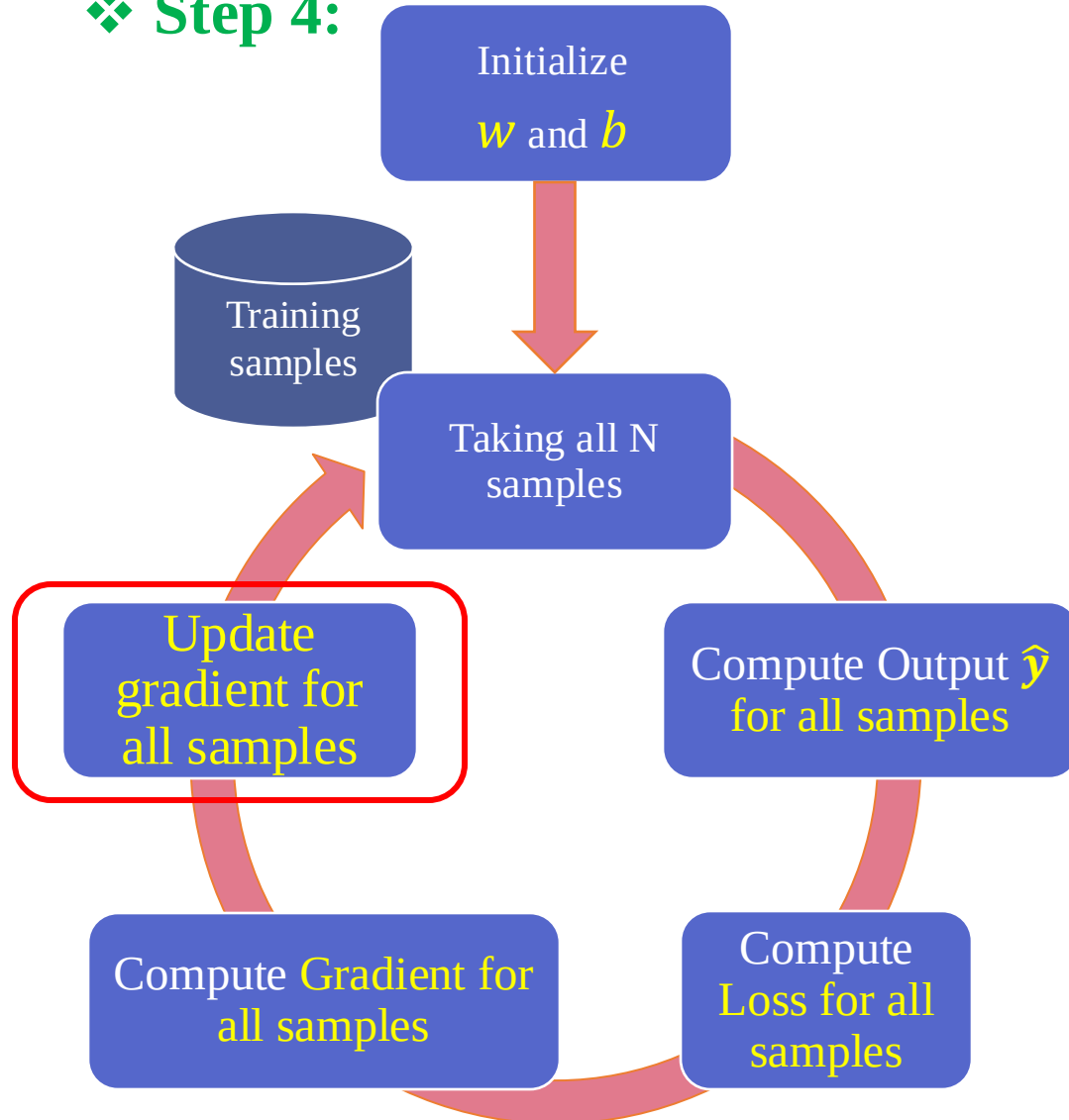
❖ Step 3:



```
1 import numpy as np
2
3 def predict(X, theta):
4     return X.dot(theta)
5
6 def compute_loss(y_hat, y):
7     N = y.shape[0]
8
9     return np.sum((y_hat - y) ** 2) / N
10
11 def compute_gradient(y_hat, y, X):
12     N = y.shape[0]
13     k = 2 * (y_hat - y)
14
15     return X.T.dot(k) / N
16
17 def update_gradient(theta, gradient, lr):
18     theta = theta - lr * gradient
19
20     return theta
```

Code Implementation

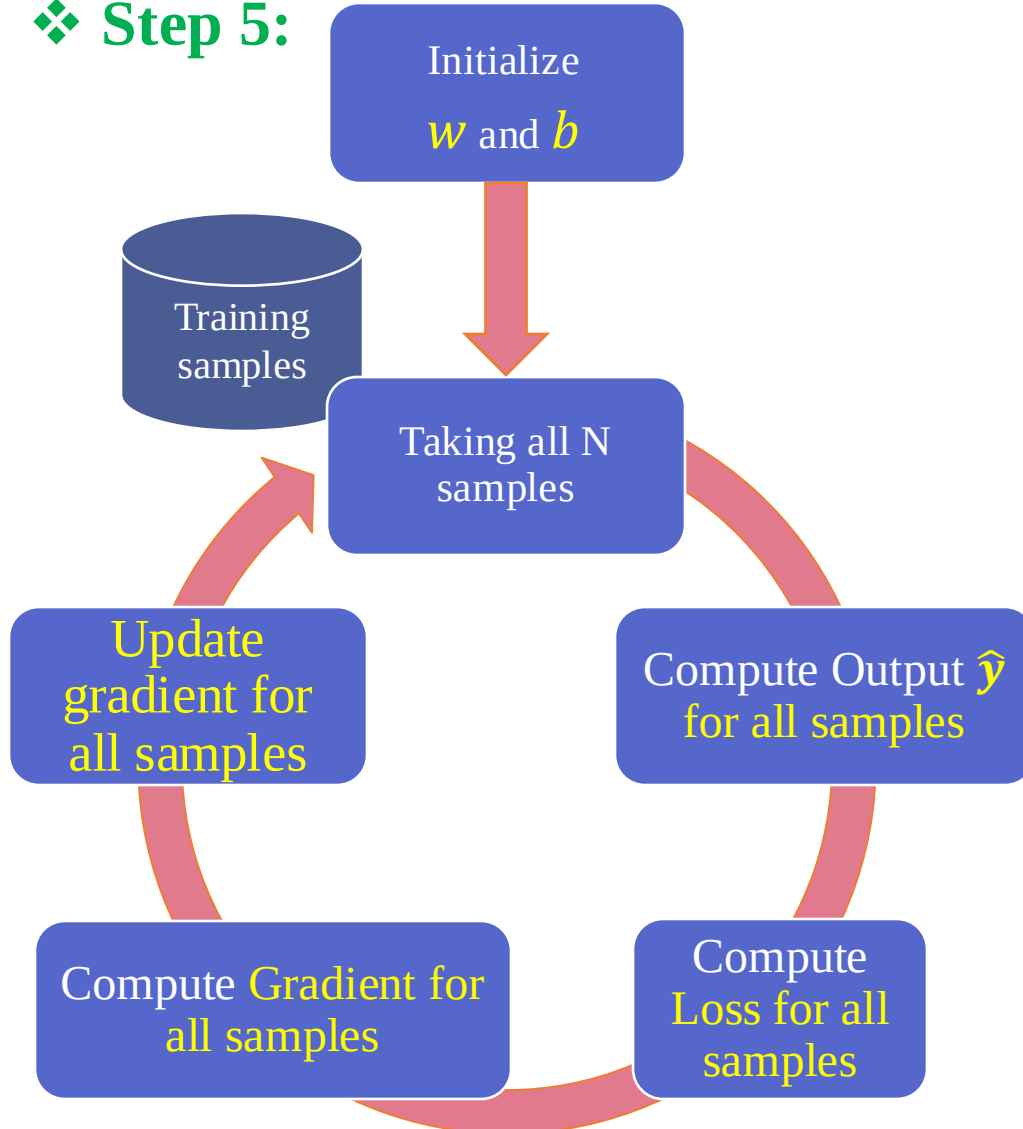
❖ Step 4:



```
1 import numpy as np
2
3 def predict(X, theta):
4     return X.dot(theta)
5
6 def compute_loss(y_hat, y):
7     N = y.shape[0]
8
9     return np.sum((y_hat - y) ** 2) / N
10
11 def compute_gradient(y_hat, y, X):
12     N = y.shape[0]
13     k = 2 * (y_hat - y)
14
15     return X.T.dot(k) / N
16
17 def update_gradient(theta, gradient, lr):
18     theta = theta - lr * gradient
19
20     return theta
```

Code Implementation

❖ Step 5:



```
1 # Các thông số huấn luyện
2 epoch_max = 4
3 lr = 0.001 # Learning rate
4 N = len(y)
5 losses = []
6
7 # Huấn luyện qua các epoch
8 for epoch in range(epoch_max):
9     # Dự đoán giá trị
10    y_hat = predict(X_bias, theta)
11
12    # Tính Loss (MSE) - Chia cho N
13    loss = compute_loss(y_hat, y)
14    losses.append(loss)
15
16    # Tính gradient
17    gradient = compute_gradient(y_hat, y, X_bias)
18
19    # Cập nhật tham số
20    theta = update_gradient(theta, gradient, lr)
21
22    print(f'Epoch {epoch+1}: Loss = {loss}, theta = {theta}')
```

Code Implementation

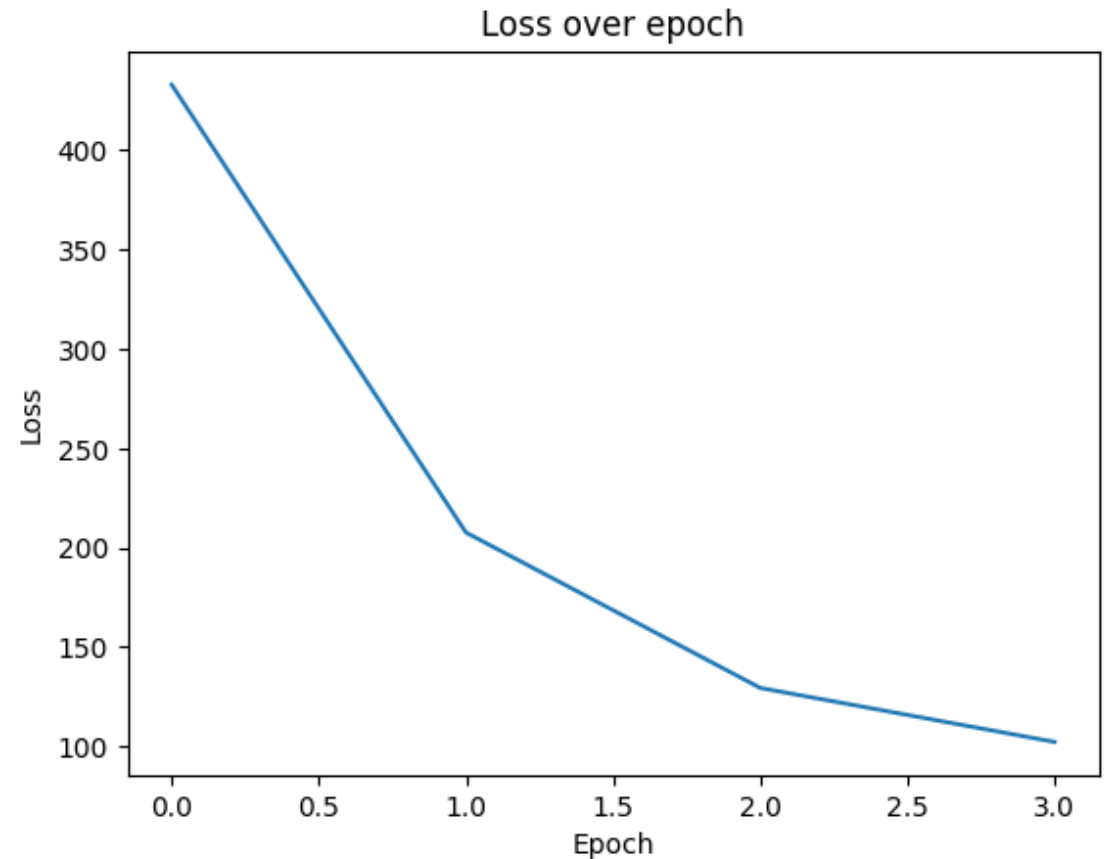
❖ Compare performance

```
1 # Before training prediction
2 X_test = np.array([[1, 4, 13]])
3 y_hat = predict(X_test, theta)
4 loss = compute_loss(y_hat, y)
5 print(f'Loss before training: {loss}')
```

Loss before training: 635.5

```
1 # After training prediction
2 X_test = np.array([[1, 4, 13]])
3 y_hat = predict(X_test, theta)
4 loss = compute_loss(y_hat, y)
5 print(f'Loss after training: {loss}')
```

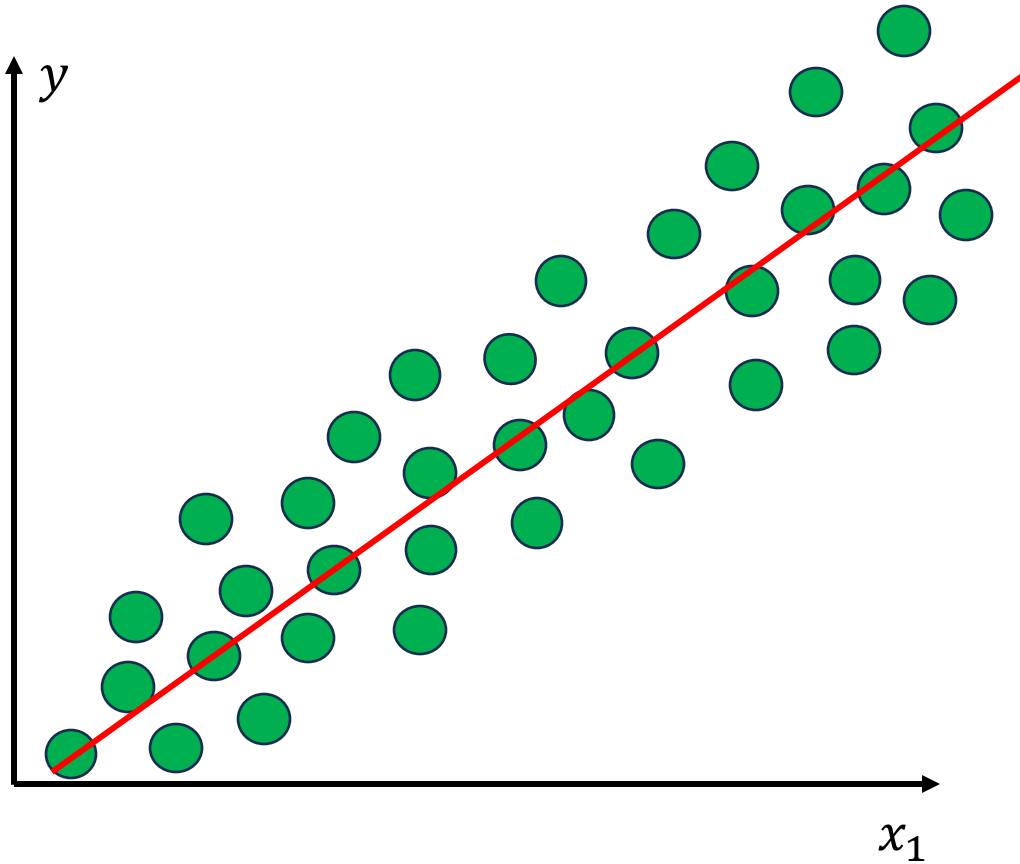
Loss after training: 239.97983168947812



Summarization and QA

Summarization

❖ Summarization



In this lecture, we have discussed:

1. Introduction to Vectorized Linear Regression.
2. Discuss about how the Vectorized Linear Regression works.
3. Implement Vectorized Linear Regression.

Question



